

UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

Minh Tri Thai

**A CONTEXT-AWARE GATED MLP FOR
ADAPTIVE MULTI-SCALE STOCK
PRICE FORECASTING**

BACHELOR'S THESIS IN INFORMATION TECHNOLOGY
STANDARD PROGRAM

Ho Chi Minh City, 07/2025

UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

Minh Tri Thai - 20120220

**A CONTEXT-AWARE GATED MLP FOR
ADAPTIVE MULTI-SCALE STOCK
PRICE FORECASTING**

BACHELOR'S THESIS IN INFORMATION TECHNOLOGY
STANDARD PROGRAM

THESIS ADVISOR

MSc. Van Quy Tran

Ho Chi Minh City, 07/2025

Commitment

We hereby certify that this thesis is the result of our independent research. All information, data, and references used in this thesis have been fully cited and properly sourced. We affirm that the content of this thesis has never been submitted for any degree or certification at any other institution of higher education. I take full responsibility for the accuracy and integrity of the content presented in this thesis.

Explanation of Thesis Modification

During the experimentation process, our group focused on improving the model's multi-timeframe mechanism through various methods and by integrating related studies, but the results were not satisfactory. However, during this process, we discovered an effective method to enhance another component of the model, which yielded very promising results. This alternative approach is the one presented in this report.

BẢN THUYẾT MINH CHÍNH SỬA BÁO CÁO ĐỀ TÀI
KHÓA LUẬN TỐT NGHIỆP

Tên đề tài : Cải thiện dự đoán xu hướng chứng khoán bằng cách tích hợp thông tin đa
khung thời gian

Sinh viên thực hiện : Thái Minh Trí, MSSV: 20120220

Hội đồng bảo vệ: Kỳ Thuật Phan Mẫn

Ngày bảo vệ: 28/07/2025

(Chúng) tôi đã hoàn chỉnh [khóa luận/ thực tập/ thực tập dự án] tốt nghiệp theo góp ý của Hội
đồng và nhận xét của Giảng viên phản biện. Nội dung đã hiệu chỉnh như sau:

1. Tên đề tài đã điều chỉnh: (nếu có)

- Tiếng Việt : Mô hình Context-Aware Gated MLP cho đa khung thời gian trong dự
đoán xu hướng thị trường

- Tiếng Anh : A Context-Aware Gated MLP for Adaptive Multi-Scale Stock Price
Forecasting

2. Điều chỉnh nội dung báo cáo:

- Đổi các trang tiếng Việt sang tiếng Anh, thống nhất ngôn ngữ bài báo cáo.
- Xóa bớt các trang thừa như phụ lục tiếng Nôm,...
- Gộp các chương ngắn, cân đối giữa các chương (cụ thể gồm chương Problem Definition
vào chương Methodology).
- Slide thêm số trang, thêm bài báo đã nộp, thêm slide minh họa cơ chế học của mô hình
StockMixer.

Xác nhận của Giảng viên hướng dẫn
(Ký tên, ghi rõ họ tên)


Hân Văn Dung

Xác nhận của Giảng viên phân biện
(Ký tên, ghi rõ họ tên)


Nguyễn Thị Minh Huyền

Tp. Hồ Chí Minh, ngày 20 tháng 08 năm 2025.

(Nhóm) Sinh viên
(ký tên, ghi rõ họ tên)


Thái Minh Trí

Bản thuyết minh chỉnh sửa báo cáo sau bảo vệ - 2

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM

Độc lập – Tự do – Hạnh phúc

ĐƠN XIN ĐỔI TÊN ĐỀ TÀI KHOA LUẬN TỐT NGHIỆP

Chương trình Chính quy Chuẩn

Kính gửi: Ban chủ nhiệm Khoa Công nghệ Thông tin

Chúng tôi gồm:

Thái Minh Trí, MSSV: 20120220, CNgành: Kỹ Thuật Phần Mềm

Chúng tôi đã đăng ký thực hiện đề tài khoá luận với tên: Cải thiện dự đoán xu hướng chứng khoán bằng cách tích hợp thông tin đa khung thời gian.

Dưới sự hướng dẫn của giảng viên: ThS. Trần Văn Quý

Nay chúng tôi xin được đổi tên đề tài thành:

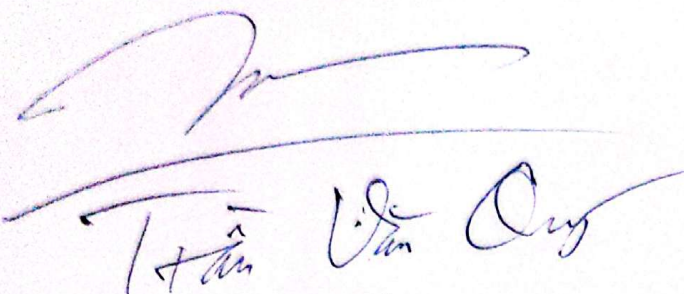
- Tên tiếng Việt: Mô hình Context-Aware Gated MLP cho đa khung thời gian trong dự đoán xu hướng thị trường
- Tên tiếng Anh: A Context-Aware Gated MLP for Adaptive Multi-Scale Stock Price Forecasting

Lý do: Trong quá trình thử nghiệm, nhóm tập trung cải tiến cơ chế đa khung thời gian của mô hình bằng nhiều phương pháp và tích hợp các nghiên cứu liên quan, nhưng vẫn không thu được kết quả khả quan. Tuy nhiên, trong quá trình thực nghiệm, nhóm đã tìm ra một phương pháp hiệu quả để cải tiến một thành phần khác của mô hình, cho kết quả rất tốt cũng chính là hướng tiếp cận được trình bày trong bài báo đã nộp.

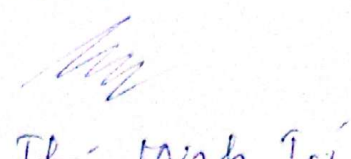
Trân trọng kính chào.

Tp. HCM, ngày tháng năm

Xác nhận của (các) Giảng viên hướng dẫn
(Ký và ghi rõ họ tên)


Trần Văn Quý

Người làm đơn
(Ký và ghi rõ họ tên)


Thái Minh Trí

Examiner's comments

Advisor's comments

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(HƯỚNG NGHIÊN CỨU)

Tên đề tài : *Mô hình Context-Aware Gated MLP cho đa khung thời gian trong dự đoán xu hướng thị trường.*

Sinh viên thực hiện : 20120220 – Thái Minh Trí

Giảng viên phân biện: TS. Nguyễn Thị Minh Tuyền

1. Chủ đề và ý tưởng nghiên cứu:

StockMixer là một kiến trúc dựa vào các khối MLP (Multi-layer Perception) được thiết kế để dự đoán giá cổ phiếu. StockMixer kết hợp giữa các khối với nhau: khối chỉ báo, khối thời gian và khối cổ phiếu. Đề tài này tập trung vào việc cải tiến StockMixer nói trên nhằm mục đích tìm kiếm kết quả tốt hơn.

2. Phương pháp nghiên cứu:

Nhóm sinh viên đã tìm hiểu các kiến thức liên quan về Mô hình hoá chuỗi thời gian nhiều thang đo (Multi-Scale Time-Series Modeling), về Mô hình hoá mối quan hệ cổ phiếu (Stock Relationship Modeling), về các mô hình dựa trên MLP và các mô hình Attention-Free. Tiếp đó, sinh viên đi sâu vào kiến trúc StockMixer, cài đặt, thực nghiệm và cải tiến mô hình đã tìm hiểu để tìm kiếm kết quả tốt hơn.

3. Đóng góp Khoa học và thực tiễn:

Đóng góp chính về mặt khoa học của Khoá luận này là cải tiến kiến trúc StockMixer bằng việc bổ sung MLP có cổng, cho phép đưa ngữ cảnh thị trường (market context) vào mô hình.

Việc thực nghiệm được tiến hành với baseline là StockMixer trên tập dữ liệu NASDAQ, S&P500 (dữ liệu đã sử dụng trong bài báo liên quan StockMixer), và bổ sung thêm 117 tài sản số (cryptocurrency). Việc thực nghiệm được tiến hành với cấu hình Kaggle's GPU T4 x2. Tuy nhiên, kết quả của baseline (Kiến trúc StockMixer) lại chênh lệch khá nhiều so với kết quả được công bố trước đó trong bài báo liên quan StockMixer. Chẳng hạn, với chỉ số IC (Information Coefficient) của NASDAQ: bài báo gốc (0.043), kết quả thực nghiệm (0.033), kết quả cải tiến (0.042). Tương tự cho kết quả của S&P 500, các chỉ số đều sai lệch khá nhiều so với bài báo gốc.

Do vậy, việc khẳng định kết quả cải tiến chưa đảm bảo được độ tin cậy. Nhóm tác giả cần kiểm tra lại vấn đề thực nghiệm của mình để có kết quả không quá sai lệch lớn với kết quả gốc trước khi tiến hành cải tiến.

4. Báo cáo viết:

Báo cáo viết (sau khi chỉnh sửa theo góp ý của phản biện) có tổng cộng 81 trang, gồm 7 chương chính. Chương 1 giới thiệu về các thử thách trong bài toán dự đoán, kiến trúc StockMixer và các đóng góp dự kiến của đề tài. Chương 2 tập trung vào các công trình liên quan. Chương 3 đề cập phương pháp StockMixer. Chương 4 định nghĩa bài toán. Chương 5 trình bày phương pháp cải tiến. Chương 6 tiến hành thực nghiệm. Chương 7 kết luận và đề xuất hướng phát triển tương lai. Nhìn chung, báo cáo được viết rõ ràng, trình bày có logic. Tuy nhiên, cần để ý cân đối lại độ dài của các chương. Cụ thể, chương 4 chỉ nêu định nghĩa bài toán trong chưa tới 1 trang, nên gộp chung vào chương khác sẽ hợp lý hơn.

5. Trình bày trước hội đồng:

Tốt.

6. Công bố khoa học/ ứng dụng thực tế:

01 bài báo đang đã được nộp, chưa có kết quả.

Đánh giá xếp loại: Giỏi-Xuất sắc

TP.HCM, ngày 28 tháng 07 năm 2025

Giảng viên phản biện

(Ký và ghi rõ họ tên)



Nguyễn Thị Minh Tuyên

Acknowledgements

We would like to express our sincere gratitude to all the lecturers of the Faculty of Information Technology, University of Science, for their dedicated teaching and support throughout our academic journey. The guidance and knowledge imparted by our instructors have laid the foundation for us to carry out and complete this thesis.

We are especially grateful to MSc. Trần Văn Quý for his close supervision and thoughtful guidance throughout the research and writing process. His knowledge and encouragement have been invaluable to us.

We also sincerely thank the members of the Thesis Committee for reviewing our report and providing constructive feedback and suggestions to enhance the quality of this thesis.

Despite our best efforts, due to our limited experience and knowledge, this thesis may still contain shortcomings in both content and presentation. We respectfully welcome any feedback and suggestions from our lecturers to help us further improve our work.

Detailed proposal



fit@hcmus

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ CƯƠNG KHOÁ LUẬN TỐT NGHIỆP

MÔ HÌNH CONTEXT-AWARE GATED MLP CHO ĐA KHUNG THỜI GIAN TRONG DỰ ĐOÁN XU HƯỚNG THỊ TRƯỜNG

*(A Context-Aware Gated MLP for Adaptive Multi-Scale Stock Price
Forecasting)*

1 THÔNG TIN CHUNG

Người hướng dẫn:

– ThS. Trần Văn Quý (Khoa Công nghệ Thông tin)

[Nhóm] Sinh viên thực hiện:

1. Thái Minh Trí (MSSV: 20120220)

Loại đề tài: Nghiên cứu

Thời gian thực hiện: Từ 01/2025 đến 07/2025

2 NỘI DUNG THỰC HIỆN

2.1 Giới thiệu về đề tài

Trong bối cảnh thị trường chứng khoán ngày càng trở nên biến động và phức tạp, dự báo giá cổ phiếu là một bài toán trọng tâm và đầy thách thức. Dữ liệu lịch sử chứa đựng nhiều tín hiệu quan trọng ở các khung thời gian khác nhau, từ những dao động ngắn hạn cho đến xu hướng dài hạn. Việc khai thác đồng thời các tín hiệu này đòi hỏi phải phân tích sâu sắc và hiệu quả, nhằm nhận diện được các đặc trưng ẩn và đưa ra dự báo chính xác. Nghiên cứu này hướng tới việc phát triển một mô hình dự báo giá cổ phiếu sử dụng kiến trúc MLP đơn giản, kết hợp với chiến lược phân đoạn dữ liệu theo đa khung thời gian. Phương pháp này giúp tận dụng tối đa các thông tin từ dữ liệu lịch sử mà không cần đến các kiến trúc phức tạp như Transformer hay GNN, từ đó giảm thiểu chi phí tính toán và tăng tính ứng dụng trong thực tiễn.

2.2 Mục tiêu đề tài

Nhận thấy tầm quan trọng, đề tài được thực hiện với những mục tiêu sau:

- Nghiên cứu và tổng hợp các phương pháp dự báo giá cổ phiếu hiện có dựa trên dữ liệu lịch sử.
- Phát triển mô hình dự báo dựa trên kiến trúc MLP, kết hợp chiến lược phân đoạn dữ liệu theo các khung thời gian khác nhau để khai thác các tín hiệu ngắn hạn và dài hạn.
- Đánh giá hiệu quả dự báo của mô hình thông qua các chỉ số đo lường tiêu chuẩn như Information Coefficient (IC), Rank Information Coefficient (RIC), Precision@N và Sharpe Ratio.
- So sánh kết quả của mô hình đề xuất với các phương pháp dự báo hiện có nhằm khẳng định hiệu quả của cách tiếp cận đơn giản nhưng mạnh mẽ.

2.3 Phạm vi của đề tài

Đề tài tập trung vào việc dự báo giá cổ phiếu dựa trên dữ liệu lịch sử từ các thị trường chứng khoán tiêu biểu như NASDAQ và S&P500. Nghiên cứu giới hạn ở khâu xử lý dữ liệu, xây dựng mô hình dự báo và đánh giá kết quả, không đi sâu vào các yếu tố kinh tế vĩ mô khác. Dữ liệu sử dụng chủ yếu bao gồm các chỉ số giá và các thông tin tài chính có sẵn, nhằm đảm bảo tính khách quan và ứng dụng rộng rãi trong phân tích thị trường.

2.4 Cách tiếp cận dự kiến

2.4.1 Phân đoạn dữ liệu theo đa khung thời gian

Nhận thấy rằng các tín hiệu thị trường có thể xuất hiện ở nhiều mức độ khác nhau, dữ liệu lịch sử được chia thành các khung thời gian ngắn hạn, trung hạn và dài hạn. Ngoài ra MLP là model còn đơn giản rất khó để học mối tương quan giữa các điểm thời gian trong 1 khung thời gian như các cơ chế attention, đặc biệt khi khung thời gian dài, thiết lập học theo đa khung thời gian sẽ giúp MLP nắm bắt các mức độ chi tiết khác nhau từng khung thời gian, ví dụ khung lớn thì học trend, khung nhỏ thì học các giao động giữa các ngày lân cận, cho phép mô hình học được các biểu diễn đặc trưng riêng biệt của từng khoảng thời gian, từ đó học được những biến động chi tiết và xu hướng ẩn trong thị trường.

2.4.2 Xây dựng mô hình dự báo dựa trên kiến trúc Stock-Mixer

Nhóm sẽ xây dựng mô hình dự báo sử dụng kiến trúc Stock-Mixer, là một mô hình vận dụng kiến trúc MLP-Mixer trong lĩnh vực thị giá máy tính để khai thác thị trường tài chính chứng khoán. Mô hình này được thiết kế với các lớp kết nối đầy đủ nhằm trộn lẫn thông tin theo cả chiều thời gian và các chỉ báo của dữ liệu. Qua đó, mô hình có khả năng học được mối quan hệ phức tạp giữa các tín hiệu trên nhiều khung thời gian khác nhau, giúp cải thiện độ chính xác của dự báo.

2.4.3 Thử nghiệm nhiều chiến lược đa khung thời gian trên StockMixer

Khám phá một số kỹ thuật và nghiên cứu liên quan để thay thế chiến lược học đa khung thời gian của Stock Mixer, bao gồm lựa chọn khung thời gian động của MSGNet thông qua FFT để xếp hạng tần số thời gian nổi bật hoặc sử dụng biến đổi wavelet (DWT) để phân tích chuỗi thời gian thành nhiều độ phân giải.

2.4.4 Cải tiến Stock-Mixer bằng cơ chế chọn lọc tính chính đặc trưng ẩn của thị trường

Để tăng cường khả năng học được các đặc trưng ẩn quan trọng của thị trường và giảm thiểu ảnh hưởng của các tín hiệu nhiễu trong thị trường, mô hình sẽ tích hợp cơ chế điều chỉnh thông tin đặc trưng ẩn. Cơ chế này giúp mô hình tự động lựa chọn những đặc trưng hữu ích từ các dữ liệu phân đoạn và điều chỉnh dòng thông tin một cách linh hoạt, từ đó tập trung vào các tín hiệu thị trường chủ yếu.

2.4.5 Huấn luyện và đánh giá mô hình

Quá trình huấn luyện mô hình sẽ được thực hiện trên tập dữ liệu đã được tiền xử lý trước và được phân đoạn khung thời gian trong mô hình. Các tham số của mô hình sẽ được tối ưu hóa qua chủ yếu thông qua thực nghiệm theo kinh nghiệm nhằm giảm thiểu sai số dự báo và chọn ra mô hình tối ưu. Hiệu quả của mô hình sẽ được đánh giá bằng các chỉ số tiêu chuẩn trong lĩnh vực tài chính, như Information Coefficient (IC), Rank Information Coefficient (RIC), Precision@N và Sharpe Ratio. Qua đó, kết quả của mô hình đề xuất sẽ được so sánh với các phương pháp dự báo hiện có.

2.5 Kết quả dự kiến của đề tài

Sau khi hoàn thiện mô hình, kết quả dự báo có thể được triển khai trong một hệ thống hỗ trợ ra quyết định đầu tư. Hệ thống cung cấp các dự báo giá cổ phiếu dựa trên dữ liệu thực tế, giúp các nhà đầu tư và chuyên gia tài chính có cái nhìn tổng quan về xu hướng thị trường và hỗ trợ quá trình ra quyết định một cách hiệu

quả.

Nghiên cứu cũng góp phần mở ra hướng tiếp cận mới trong lĩnh vực dự báo tài chính, với tiềm năng ứng dụng rộng rãi trong các hệ thống phân tích và dự báo giá cổ phiếu

2.6 Kế hoạch thực hiện

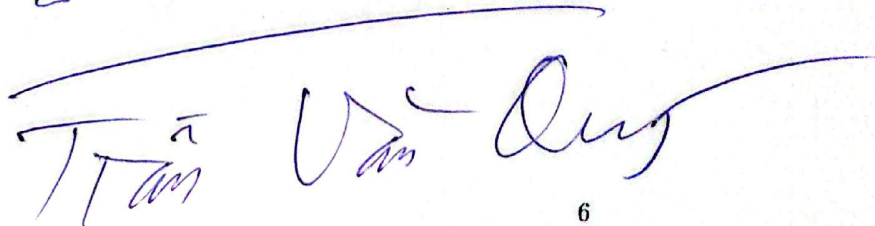
Dựa trên phạm vi và mục tiêu của đề tài, kế hoạch thực hiện đề tài được phân chia như sau:

STT	Thời gian	Công việc
1	Tháng 01/2025	Giới hạn phạm vi đề tài, xác định đối tượng nghiên cứu, tìm hiểu các phương pháp tiếp cận, các công việc/ bài báo liên quan và chọn bài báo để cải tiến.
2	02/2024	Khảo sát và tiếp cận đối tượng. Tìm hiểu các phương pháp, mô hình có thể nghiên cứu
3	03/2024	Viết và cập nhật đề cương khóa luận.
4	03,04,05/2025	Thực nghiệm và tiếp tục cải tiến
5	04,05,06/2025	Tiến hành viết báo cáo theo kết quả thử nghiệm
6	Tháng 07/2025	Hoàn thành các tài liệu liên quan khoá luận

Tài liệu

- [1] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, pp. 11121–11128, Jun. 2023.
- [2] X. Liu and W. Wang, "Deep time series forecasting models: A comprehensive survey," *Mathematics*, vol. 12, no. 10, 2024.
- [3] J. Fan and Y. Shen, "Stockmixer: A simple yet strong mlp-based architecture for stock price forecasting," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 8389–8397, Mar. 2024.
- [4] G. Liu, Y. Mao, Q. Sun, H. Huang, W. Gao, X. Li, J. Shen, R. Li, and X. Wang, "Multi-scale two-way deep neural network for stock trend prediction," in *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, pp. 4555–4561, 2021.
- [5] W. Cai, Y. Liang, X. Liu, J. Feng, and Y. Wu, "Msgnet: Learning multi-scale inter-series correlations for multivariate time series forecasting," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 11141–11149, Mar. 2024.

XÁC NHẬN
CỦA NGƯỜI HƯỚNG DẪN
(Ký và ghi rõ họ tên)



Trần Văn Dũng

TP. Hồ Chí Minh, ngày... tháng... năm...
NHÓM SINH VIÊN THỰC HIỆN
(Ký và ghi rõ họ tên)


Trần Minh Trí

Table of Contents

Explanation of Thesis Topic Modification	ii
Advisor’s Comments	vii
Examiner’s Comments	viii
Acknowledgements	xi
Thesis Proposal	xii
Table of Contents	xix
List of images	xxii
List of tables	xxiii
Abstract	xxiii
1 Introduction	1
1.1 Challenges	2
1.2 Inspiration	2
1.3 Further Evaluation	3
1.4 Contributions	3
2 Related works	5
2.1 Multi-Scale Time-Series Modeling	5

2.1.1	Static timeframes strategy	5
2.1.2	Dynamic timeframes strategy	8
2.2	Stock Relationship Modeling	9
2.2.1	Hypergraph	9
2.2.2	Adaptive Multi-hop GNN	11
2.3	MLP-Based and Attention-Free Models	12
3	StockMixer methodologies	15
3.1	Multi-time frame segmentation strategy	15
3.2	Indicator Mixing	17
3.3	Time Mixing	17
3.4	Stock Mixing	18
3.5	Performance	19
4	Methodology	21
4.1	Problem definition	21
4.2	Model overview	22
4.2.1	Input representation	22
4.2.2	Indicators Used for Internal Gating Instead of Input Prices Augmentation	22
4.2.3	Objective output	22
4.3	Extracting market context	24
4.4	Learning selectivity and modulating markets via a gating mechanism	25
4.5	Refined Gated MLP	27
4.6	Enhance StockMixer approach	31
5	Experiments	33
5.1	Multi-timeframe	33
5.1.1	LagMixer: Embedding via Temporal Lag	33
5.1.2	DWTMixer: Wavelet-Based Multi-Scale Embedding	35
5.1.3	Dynamic Scale-Aware Embedding	36

5.1.4	Multi-Scale Weighting	36
5.2	Datasets	37
5.2.1	Benchmarks	37
5.2.2	Dataset Sources	38
5.2.3	Cryptocurrency Preprocessing	38
5.3	Baselines	41
5.3.1	StockMixer as primary baseline	41
5.3.2	Additional state of the art models	42
5.4	Metrics	42
5.5	Implementation Details	43
5.6	Overall comparison	44
5.6.1	Re-evaluation of StockMixer	44
5.6.2	Results	45
5.6.3	Discussion and Remarks	45
5.7	Ablation study	46
5.7.1	Model Component	46
5.7.2	Effect of ablated variants	48
6	Conclusion And Future Works	50
6.1	Conclusion	50
6.2	Future Works	51
	Submitted Papers	53
	References	54

List of images

2.1	The Multi Scale Two Way DNN framework	6
2.2	The Multi-DQN framework	7
2.3	The MSGNet framework	8
2.4	The STHAN-SR framework	10
2.5	The DLinear framework	13
2.6	The TSMixer framework	14
3.1	The StockMixer framework	16
3.2	Triangular mask	18
3.3	Performance metrics reported by StockMixer	20
4.1	Stacked gated MLP for StockMixer	23
4.2	Gated multilayer perceptron architecture	26
4.3	Squeeze-and-Excitation Block	27
4.4	Novel gated MLP	28
4.5	Gated MLP integration for StockMixer	31
5.1	Experimental LagMixer	34
5.2	Experimental DWTMixer	35
5.3	Experimental Dynamic Scale-Aware Embedding	36
5.4	Cryptocurrency date distribution	39
5.5	Ablated gated MLP	47

List of tables

5.1	NASDAQ metrics for Dynamic Scale–Aware Embedding, Wavelet-Based Multi-Scale Embedding and Multiscale weighting . .	34
5.2	Detailed information on the number of assets, start and end dates, as well as the training, validation, and test durations for each dataset.	37
5.3	Data quality metrics for the cryptocurrency.	38
5.4	Statistical summary and validation metrics for price-related columns.	39
5.5	Empirically tuned hyperparameter settings across NASDAQ, S&P 500, and Crypto datasets.	43
5.6	Comparison of models across NASDAQ, S&P 500, and Crypto datasets. Metrics: Information Coefficient (IC), Rank IC (RIC), precision@10 (prec@10), and Sharpe Ratio (SR). . .	44
5.7	Comparison of Baseline, No Context, and With Context settings across NASDAQ, S&P 500, and Crypto datasets. Metrics: Information Coefficient (IC), Rank IC (RIC), precision@10 (prec@10), and Sharpe Ratio (SR).	47

Abstract

Stock price forecasting is central to quantitative finance, underpinning portfolio allocation, risk management, and algorithmic trading. Effective prediction requires modeling not only individual price dynamics but also the propagation of information across multiple temporal scales—from intraday order-flow imbalances to daily news-driven moves and weekly macroeconomic cycles.

Also, Stock price forecasting requires modeling complex inter-asset relationships across multiple time horizons, yet many state-of-the-art models such as Transformers and Graph Neural Networks suffer from high computational costs. To address this, we introduce **Context-Aware Gated MLP (CAG-MLP)**, a novel lightweight architecture that enhances multi-scale stock prediction by integrating market context information into gated MLP blocks.

Our key innovation lies in the *dual-branch gated mechanism* that dynamically modulates information flow based on precomputed market features. We extract five complementary market context features—mean return, momentum slope, realized volatility, dispersion, and PCA ratio—from each 16-day window and integrate them into gated MLP blocks. The trunk branch processes asset-specific features while the gating branch, conditioned on market context, learns to selectively emphasize or suppress latent market signals.

We validate our approach by enhancing the *StockMixer* framework, replacing its simple stock mixing module with our context-aware gated MLP

architecture. Extensive experiments on three diverse datasets (NASDAQ: 1,026 stocks, S&P 500: 474 stocks, and Cryptocurrency: 117 coins) demonstrate significant performance improvements. Our model achieves consistent gains of 27% in IC and 22% in RIC on NASDAQ, 86% in IC and 86% in RIC on S&P 500, and 22% in IC and 11% in RIC on cryptocurrency markets compared to the baseline StockMixer.

Ablation studies confirm the critical role of market context integration, showing substantial performance degradation when removing context features. Our approach maintains computational efficiency while capturing subtle market dynamics, making it particularly effective in volatile market conditions. The modular design enables easy integration into existing MLP-based pipelines, offering a practical solution for real-world financial forecasting applications. The official code is available at our ¹github.

A vietnamese version is available at: https://docs.google.com/document/d/1rmG_axZC1hr2sqkt3o2HOBGM_bX2PWdaeGMbS5fsU3M/edit?usp=sharing

¹<https://github.com/teri1712/Context-Aware-gMLP-for-Multi-Scale-Stock-Forecasting>

Chapter 1

Introduction

Stock price movements are governed by patterns unfolding across multiple temporal horizons. Short-term signals, such as intraday volatility and recent momentum shifts, often reflect immediate market sentiment and liquidity changes, while long-term trends provide insights into broader economic conditions and fundamental valuations. To address this, many multi-scale time-series models employ this approach by decomposing input sequences to capture patterns at different granularities.

For example, some models use wavelet transforms and hierarchical downsampling in multi-scale two-way deep networks [1], while others, such as MSGNet [2], utilize frequency-based decomposition combined with graph convolution to dynamically select the most informative temporal scales.

As for stock correlations, some models adopt graph-based or attention-based mechanisms to explicitly encode inter-stock dependencies and temporal relationships. One remarkable approach among them is the use of hypergraph attention networks, which capture higher-order structural information among stocks [3], along with transformer-based models that leverage self-attention to highlight significant patterns across temporal and cross-asset dimensions.

1.1 Challenges

Despite the considerable improvements in forecasting accuracy achieved by graph-based and self-attention architectures, these methods frequently entail intricate model designs and substantial computational burdens, which can impede their adoption in latency-sensitive, large-scale financial systems. In contrast, emerging evidence suggests that streamlined, fully connected networks can rival more complex frameworks while maintaining efficiency.

For instance, N-BEATS [4] exemplifies an ultra-lightweight design, relying exclusively on stacked multilayer perceptron residual blocks and omitting any convolutional, attention, or graph-based components.

Likewise, StockMixer [5] adapts MLP-Mixer to the multi-asset setting by alternating channel, temporal, and stock mixing to capture both intra and inter series patterns via simple linear projections.

1.2 Inspiration

StockMixer is a novel MLP-based architecture designed for stock movement forecasting, drawing inspiration from the MLP-Mixer framework originally proposed for vision tasks. Unlike conventional models relying on recurrent, attention-based, or graph-based mechanisms, StockMixer employs a tri-dimensional mixing strategy to capture indicator level, temporal with **causal masking and multi-scale temporal patching** to model sequential dependencies, and inter stock dependencies via lightweight feed forward layers. By extending the core principles of MLP-Mixer to the financial domain with task-specific adaptations, StockMixer demonstrates the efficacy of pure MLP architectures for multivariate time series modeling, but also enhances generalization, achieving superior empirical results across multiple stock market benchmarks in both predictive accuracy and financial metrics.

Inspired by the StockMixer study, we introduce a gated MLP (gMLP) blocks to enhance the model’s inter-stock relationships, leveraging its multi-horizon embedding representation as input to dynamically modulate cross-asset information flow. While StockMixer’s stock mixing module relies on a single hidden layer MLP, which limits its expressive power, our approach replaces this simple layer with stacked gMLP blocks-augmented with residual connections that learn market states conditioned on precomputed window-level context features.

This richer market hidden representation drives an internal gating mechanism that dynamically modulates information flow, enabling the model to capture subtle behaviors such as periodic cycles and regime shifts, while preserving architectural simplicity and computational efficiency.

While our work builds on the original StockMixer architecture, we focus solely on enhancing the stock-mixing module. The indicator and temporal mixing layers are retained from the baseline. Our contribution lies in augmenting the stock interaction phase using gated MLP blocks conditioned on market context.

1.3 Further Evaluation

Besides evaluating our method against state-of-the-art models, including StockMixer, on the NASDAQ and S&P 500 datasets, we assemble a cryptocurrency corpus of 117 coins spanning 1,035 trading days to probe cross-domain generalization. Our approach outperforms these benchmarks in both equity and cryptocurrency environments, underscoring its enhanced robustness and wide applicability.

1.4 Contributions

Our contributions are as follows:

1. Computing a market context for each window by extracting a variety of features, such as mean return and momentum slope. These features are used in conjunction with the gated MLP to enrich the information context of a specific market window, enabling more informed modulation within the mixing blocks.
2. We introduce a gated MLP-based architecture for stock modeling, enabling selective feature learning and deep hierarchical representation through residual connections. Despite its simplicity, the model captures key advantages of attention- and graph-based approaches while maintaining lower computational cost.
3. Providing evidence via an evaluation showing that incorporating context information and gated MLP improves predictive performance.

In the following sections, we formalize the problem setting, describe the proposed methodology in detail, and position our approach relative to related work such as MTDNN [1], MSGNet [2] , and STHAN-SR [3].

Chapter 2

Related works

2.1 Multi-Scale Time-Series Modeling

2.1.1 Static timeframes strategy

Multi Scale Two Way

Early multi-scale methods decompose price series into multiscale components—e.g., The Multi Scale Two Way DNN (MTDNN) [1] explicitly constructs *multi-time frame* representations of a raw price series $x \in \mathbb{R}^{T \times d}$ via two complementary pathways:

- **Wavelet-based decomposition:** A discrete wavelet transform (DWT) recursively decomposes each feature dimension $x_{:,i}$ into approximation a^m (low-frequency, long-term) and detail d^m (high-frequency, short-term) components at levels $m = 1, \dots, M$:

$$a_n^m = \sum_t h[2n - t] a_t^{m-1}, \quad d_n^m = \sum_t g[2n - t] a_t^{m-1},$$

where $a^0 = x_{:,i}$, and h, g are low-/high-pass filters. The resulting multi-scale vectors $\{d^M, \dots, d^1, a^1\}$ are concatenated into $v_i \in \mathbb{R}^{(M+1) \times T/2^M}$ and ensembled by an XGBoost regressor to yield short- and long-term trend scores s^{wavelet} .

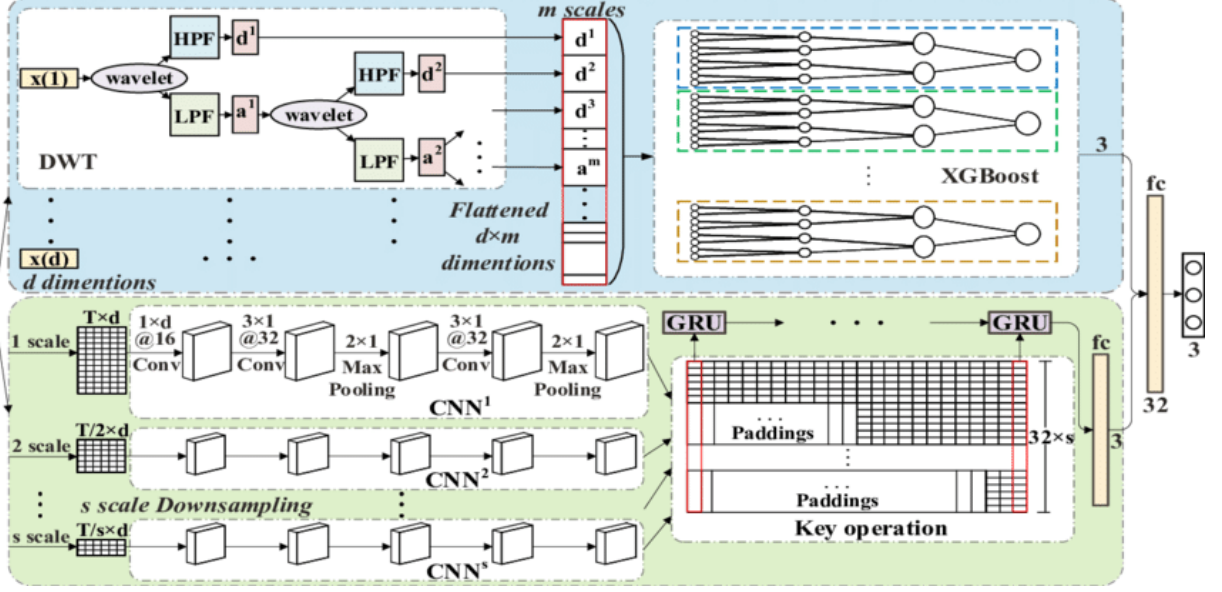


Figure 2.1: The Multi Scale Two Way DNN framework

- **Downsampling-based framing:** Temporal downsampling at scales $\{s_1, \dots, s_K\}$ produces subsequences

$$x^{(s_k)} = \{x_t \mid t \equiv 0 \pmod{s_k}\}, \quad k = 1, \dots, K.$$

Each $x^{(s_k)}$ is processed by a small convolutional network to extract spatial features, then the sequence of feature vectors is temporally cascaded via a GRU to produce s^{sample} .

Finally, the two pathways are fused by a fully connected “mixture” layer:

$$\hat{y} = \text{SoftMax}(W[s^{\text{wavelet}} \parallel s^{\text{sample}}] + b),$$

allowing the model to jointly attend to orthogonal frequency- and time-resolution cues. This parallel multi-time frame strategy improves robustness to noise and captures both rapid and gradual price dynamics [1].

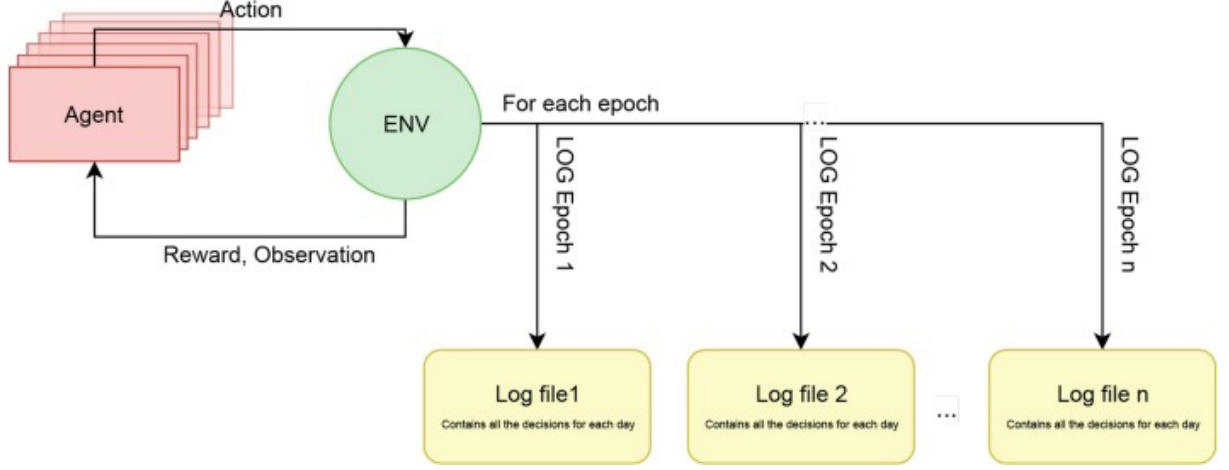


Figure 2.2: The Multi-DQN framework

Multi-Resolution State Representation in Multi-DQN

To address the challenge of capturing market dynamics across various time scales, MultiDQN [6] propose a multi-resolution input sampling technique for their ensemble of Deep Q-Learning agents. This approach constructs the state vector for the learning agent by concatenating time-series data from three distinct temporal granularities: **hourly, daily, and weekly**.

Specifically, the input vector $S(t)$ at a given time t is formed by the concatenation of the last l hourly observations, the last m daily aggregated observations, and the last n weekly aggregated observations. The authors empirically set these values to the last 40 hours, 20 days, and 8 weeks.

This strategy is designed to provide the agent with a comprehensive market view, capturing immediate, short-term fluctuations through hourly data while simultaneously considering longer-term trends and patterns via daily and weekly aggregates. By encoding information from different resolutions into a single state representation, the reinforcement learning agents can potentially learn more robust trading policies that are sensitive to both short-term volatility and overarching market trends.

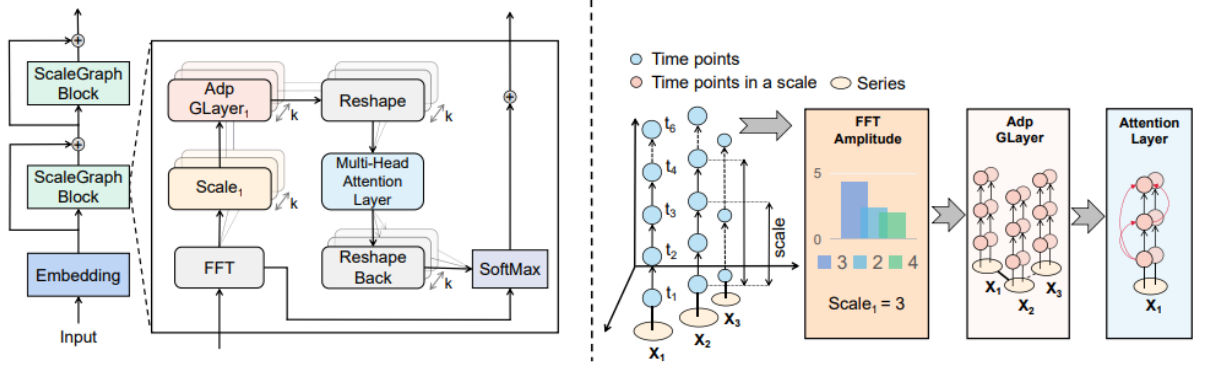


Figure 2.3: The MSGNet framework

2.1.2 Dynamic timeframes strategy

To automatically uncover salient periodicities, MSGNet first projects the input embedding $X_{\text{emb}} \in \mathbb{R}^{d_{\text{model}} \times L}$ into the frequency domain via Fast Fourier Transform (FFT) and computes its average amplitude:

$$F = \text{Avg}(\text{Amp}(\text{FFT}(X_{\text{emb}}))) , \quad F \in \mathbb{R}^L .$$

The top- k frequencies $\{f_i\}_{i=1}^k$ are then selected by

$$f_1, \dots, f_k = \arg \text{TopK}_{f \in \{1, \dots, \lfloor L/2 \rfloor\}} F_f ,$$

and converted into time scales

$$s_i = \frac{L}{f_i} , \quad i = 1, \dots, k .$$

This data-driven detection yields a set of dynamic scales $\{s_i\}$ that reflect the most prominent periodic behaviors in the series. Each scale s_i is used to reshape and segment the input into a tensor $X^{(i)} \in \mathbb{R}^{d_{\text{model}} \times s_i \times f_i}$, enabling subsequent modules—adaptive multi-hop graph convolutions for inter-series correlation and multi-head self-attention for intra-series dependencies—to operate on temporally coherent segments. By coupling

frequency-derived scale selection with learnable graph and attention layers, MSGNet flexibly adapts to evolving market rhythms and captures both local and global dynamics in a unified framework [2].

Our work also explores the multi-scale aspect of StockMixer, but unlike wavelet or FFT-based methods, they do not perform an explicit transform. Instead, they learn the multi-scale representation directly by splitting the input sequence into patches, embedding and processing them with MLP-Mixing layers, letting it implicitly figure out the important patterns at each scale.

2.2 Stock Relationship Modeling

In financial markets, individual equities exhibit interdependencies driven by shared economic factors, sector-specific trends, and concurrent macroeconomic events. To capture these complex interactions, a variety of graph neural network (GNN) techniques have been developed that explicitly model inter-stock relationships. By treating stocks as vertices in a graph and encoding temporal and spatial dependencies through learned edge representations, these methods aim to enhance predictive accuracy and portfolio selection efficiency.

2.2.1 Hypergraph

A notable instantiation of this paradigm is STHAN-SR [3], which recasts stock forecasting as a learning-to-rank task over a **hypergraph** structure. STHAN-SR constructs a *spatio-temporal hypergraph* $\mathcal{G} = (V, E, W)$ to model higher-order dependencies among N stocks. Each vertex $v_i \in V$ corresponds to a stock, while each hyperedge $e \in E$ encodes a set of stocks sharing a common relation—e.g., membership in the same industry or linked via corporate ownership—assigned a weight $w(e)$ to reflect its significance. Formally, the hypergraph is represented by an incidence

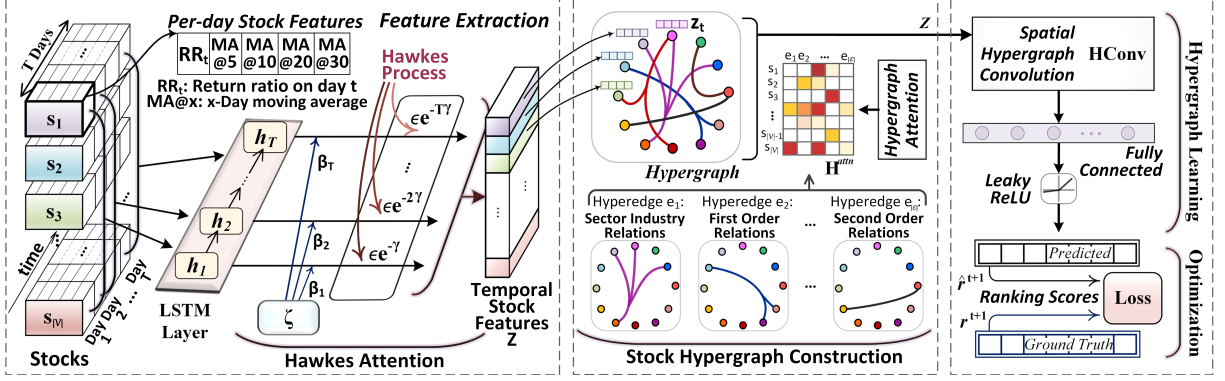


Figure 2.4: The STHAN-SR framework

matrix $H \in \{0, 1\}^{N \times |E|}$, where

$$H_{i,e} = \begin{cases} 1, & \text{if } v_i \in e, \\ 0, & \text{otherwise,} \end{cases}$$

and diagonal degree matrices D_v and D_e capture node and hyperedge degrees, respectively [3].

To propagate information across these higher-order relations, STHAN-SR applies *hypergraph convolution* layers that generalize graph convolutions to hyperedges, updating node features $X^{(l)} \in \mathbb{R}^{N \times F^{(l)}}$ via

$$X^{(l+1)} = \sigma(D_v^{-1/2} H W D_e^{-1} H^\top D_v^{-1/2} X^{(l)} P^{(l)}),$$

where W is the diagonal matrix of hyperedge weights, $P^{(l)}$ is a learned weight matrix for the feature transformation, and σ is a non-linear activation function like ELU. To further distinguish the relative importance of each hyperedge for a given node, a *hypergraph attention* mechanism is employed. This mechanism learns a new weighted incidence matrix $\hat{H}$ that replaces H in the convolution. The attention coefficients α_{ie} , representing

the importance of hyperedge e to node i , are computed as:

$$\alpha_{ie} = \frac{\exp(\text{LeakyReLU}(\vec{a}^\top [\mathbf{W}_a \mathbf{x}_i \parallel \mathbf{W}_a \mathbf{y}_e]))}{\sum_{k \in \mathcal{N}_i} \exp(\text{LeakyReLU}(\vec{a}^\top [\mathbf{W}_a \mathbf{x}_i \parallel \mathbf{W}_a \mathbf{y}_k]))},$$

where $\mathcal{N}_i$ is the set of hyperedges connected to node i , $\mathbf{y}_e$ is the feature representation of hyperedge e (obtained by averaging the features of its constituent nodes), $\mathbf{W}_a$ is a shared learnable weight matrix, $\vec{a}$ is a weight vector, and $\parallel$ denotes concatenation. The learned incidence matrix is then defined as $\hat{H}_{ie} = \alpha_{ie}$ if $H_{ie} = 1$ and 0 otherwise. This combination of hypergraph convolution and attention enables STHAN-SR to capture both local and global relational cues for robust stock ranking.

2.2.2 Adaptive Multi-hop GNN

Another notable method, MSGNet [2], utilizes adaptive multi-hop graph convolutions across multiple temporal scales to model inter-series correlations in multivariate time series forecasting. For each detected time scale s_i , the embedded tensor

$$X^{(i)} \in \mathbb{R}^{d_{\text{model}} \times s_i \times f_i}$$

is first projected back to the original variable space:

$$H_i = W_i X^{(i)}, \quad W_i \in \mathbb{R}^{N \times d_{\text{model}}}, \quad (2.1)$$

yielding $H_i \in \mathbb{R}^{N \times s_i \times f_i}$.

To uncover inter-series correlations at scale s_i , two learnable matrices $E_i^1, E_i^2 \in \mathbb{R}^{N \times h}$ are combined into an adaptive adjacency matrix:

$$A_i = \text{SoftMax}(\text{ReLU}(E_i^1 (E_i^2)^\top)) \in \mathbb{R}^{N \times N}, \quad (2.2)$$

where the SoftMax is applied row-wise to ensure normalized, sparse edge

weights.

Rather than restricting propagation to immediate neighbors, MSGNet employs the MixHop convolution operator to aggregate information from multi-hop neighborhoods up to a set of powers P :

$$H_i^{\text{out}} = \sigma\left(\parallel_{j \in P} (A_i)^j H_i\right), \quad (2.3)$$

where $\parallel$ denotes concatenation across hop orders and σ is a nonlinear activation function.

Each multi-hop output is then re-projected into the model dimension via a small MLP and fused across scales with an amplitude-weighted mixture of experts:

$$\hat{X}_i^{\text{out}} = \text{MLP}(H_i^{\text{out}}), \quad (2.4)$$

$$\alpha_i = \text{SoftMax}(F_{s_1}, \dots, F_{s_k}), \quad (2.5)$$

$$\hat{X}_{\text{out}} = \sum_{i=1}^k \alpha_i \hat{X}_i^{\text{out}}, \quad (2.6)$$

where F_{s_i} is the FFT-derived amplitude score for scale s_i . This adaptive fusion ensures that the most informative temporal scales contribute proportionally to the final representation, enabling explainable, efficient multi scale relational learning [2].

2.3 MLP-Based and Attention-Free Models

In recent years, alongside the evolution of sophisticated architectures, research in time-series forecasting has revealed that much simpler models can rival or even outperform Transformers and GNNs [7]. For example, Zeng et al. [8] showed that you don't need self-attention for long-term forecasting by introducing DLinear, which employs a straightforward linear projection on decomposed series components and surpasses numerous

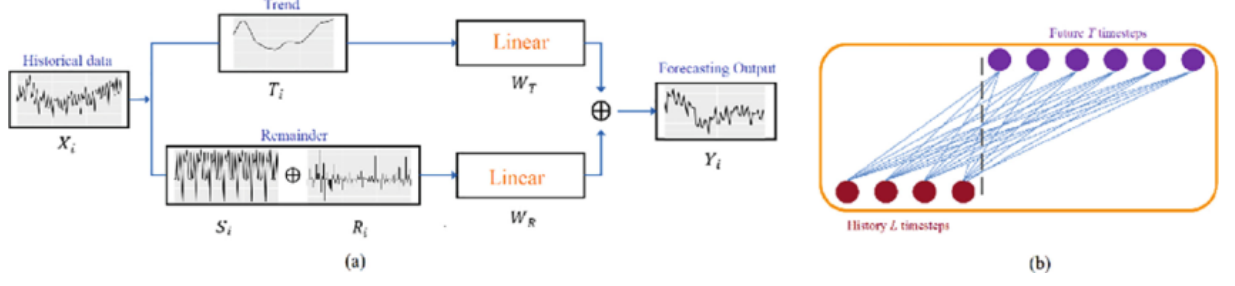


Figure 2.5: The DLinear framework

Transformer variants on standard benchmarks.

Inspired by the MLP-Mixer paradigm in computer vision—which alternates token mixing and channel mixing MLP layers to learn spatial dependencies—researchers have extended this concept to temporal data. TSMixer [9] factorizes its MLP layers along temporal and feature dimensions: temporal-mixing modules learn dependencies across time steps for each feature, and feature-mixing modules capture cross-feature interactions at each timestamp. By interleaving these operations, TSMixer models both sequential dynamics and inter-variable structure without recourse to convolution or self-attention, thereby reducing architectural complexity while preserving expressive capacity.

Within financial forecasting, purely MLP-based frameworks have also exhibited strong performance. StockMixer [5] applies mixer layers to multi resolution patches of price and technical indicator data combining short, medium, and long horizon aggregates and fuses temporal and feature representations via alternating MLP stages. This attention-free ensemble achieves competitive accuracy on intraday stock-trend classification tasks, underscoring that carefully designed MLP architectures can effectively capture the hierarchical and cross-series dependencies inherent in financial time series.

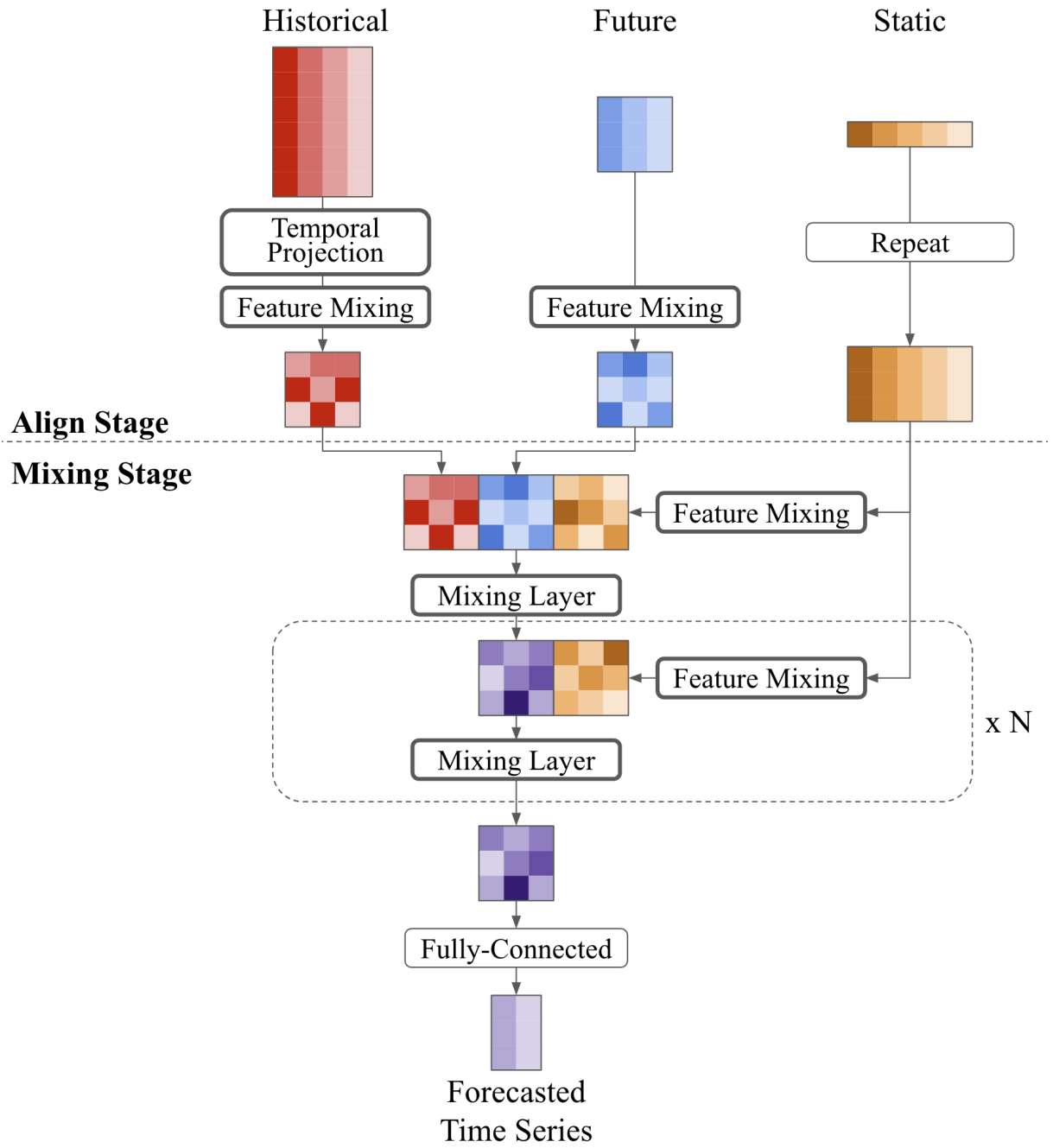


Figure 2.6: The TSMixer framework

Chapter 3

StockMixer methodologies

3.1 Multi-time frame segmentation strategy

To effectively model the non-stationary and multi-scale dynamics of financial time series, they segment the input sequence into a set of temporally pooled 'patches' at multiple compression rates. This hierarchical view allows the network to capture both short-term fluctuations and long-term trends, while reducing sensitivity to transient noise.

Given the raw input sequence

$$\mathbf{X} \in \mathbb{R}^{T \times F},$$

where T denotes the lookback window length and F the number of financial indicators, we partition $\mathbf{X}$ into subsequences (or patches) at multiple temporal scales.

For each patch size k is a power of 2, temporal pooling is applied to compress the subsequence into a lower-resolution representation:

$$\mathbf{x}^{(k)} \in \mathbb{R}^{T_k \times F},$$

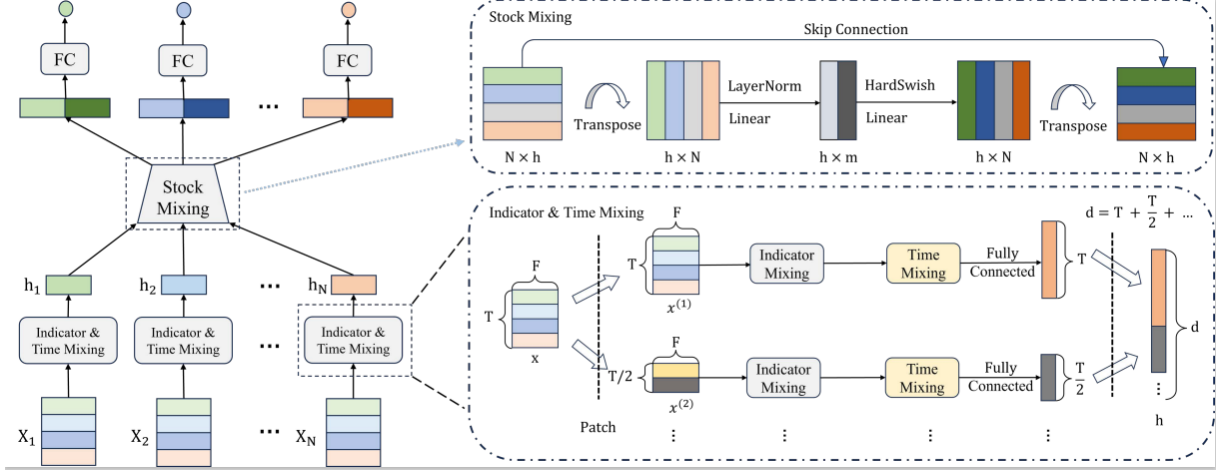


Figure 3.1: The StockMixer framework

where T_k denotes the effective sequence length after pooling,

$$T_k = \frac{T}{k}$$

Each compressed representation is then processed through the indicator mixing and time mixing modules, yielding corresponding embeddings:

$$\mathbf{h}^{(k)} \in \mathbb{R}^{T_k \times F}.$$

These embeddings are concatenated and transformed by a fully connected layer to form the aggregated temporal representation:

$$\mathbf{h} = \text{Concat} \left(\mathbf{h}^{(1)}, \mathbf{h}^{(2)}, \dots, \mathbf{h}^{(k)} \right) \in \mathbb{R}^d,$$

with $d = T + \frac{T}{2} + \dots + \frac{T}{k}$.

3.2 Indicator Mixing

Historical stock prices have been shown to serve as strong indicators of future stock trends, yet prior works often treat each financial indicator sequence independently, overlooking cross-indicator correlations. To address this, they adopt an MLP-based mixing on the indicator dimension by first transposing the time and indicator axes of the embedding. Formally, given the indicator embedding for a single stock:

$$X \in \mathbb{R}^{T \times F}, \quad (3.1)$$

where T is the lookback window length and F the number of indicator channels, they define its transpose

$$X^T \in \mathbb{R}^{F \times T}. \quad (3.2)$$

Indicator mixing is then performed as:

$$\hat{X}^T = X^T + W_2 \sigma(W_1 \text{LayerNorm}(X^T)), \quad (3.3)$$

where $W_1 \in \mathbb{R}^{H_i \times F}$ and $W_2 \in \mathbb{R}^{F \times H_i}$ are learnable weights, H_i the hidden dimension, and σ an activation function. The resulting mixed output $\hat{X} \in \mathbb{R}^{T \times F}$ is then passed to the subsequent time mixing module.

3.3 Time Mixing

The time mixing module subsequently aggregate these features across multiple temporal scales by segmenting the time series into subsequences and employing a modified MLP with an upper triangular weight mask to enforce a causal structure, as illustrated in Figure 3.2. Formally, given the

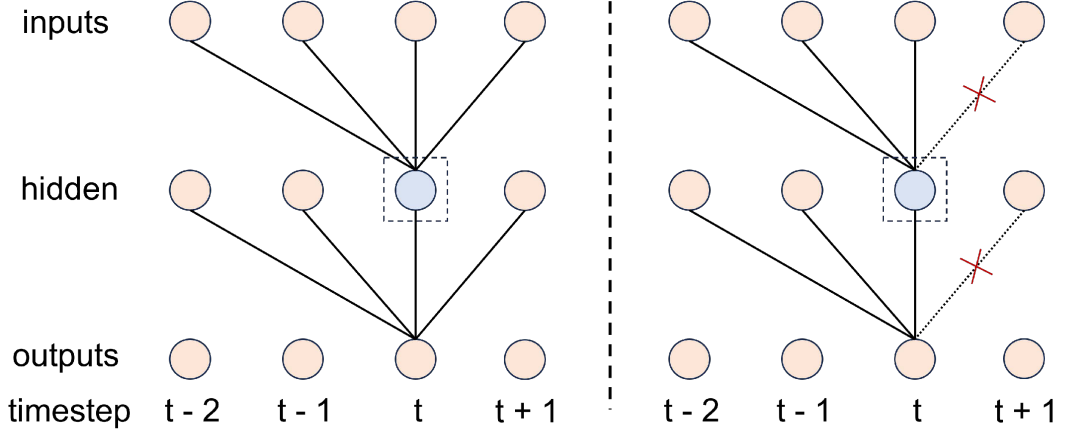


Figure 3.2: Triangular mask

mixed embedding for a single scale:

$$H \in \mathbb{R}^{L \times D}, \quad (3.4)$$

where L is the subsequence length and D the feature dimension after indicator mixing, they apply a masked MLP defined by:

$$\tilde{H} = H + \text{MaskMLP}(\text{LayerNorm}(H)), \quad (3.5)$$

with

$$\text{MaskMLP}(Z) = Z W_2^\Delta \sigma(Z W_1), \quad (3.6)$$

where $W_1 \in \mathbb{R}^{D \times H_t}$ and $W_2^\Delta \in \mathbb{R}^{H_t \times D}$ are learnable weights, H_t is the hidden dimension, σ an activation, and W_2^Δ is constrained to be upper triangular. This mask ensures that each output timestep attends only to current and past inputs, preserving causality and preventing information leakage from future steps.

3.4 Stock Mixing

After obtaining the temporal representations of all N assets, denoted as $H \in \mathbb{R}^{N \times h}$, the model constructs inter-stock relations using a dedicated

Stock Mixing block[cite: 205]. This process, illustrated in Figure 1 of the paper, deviates from the description in the main text. The actual architecture operates as follows:

The input tensor H is first transposed to allow mixing across the stock dimension. This transposed tensor is then processed by an MLP block, which consists of a linear layer that compresses the N stocks into m latent market states, a LayerNorm layer, a HardSwish activation function, and a second linear layer that projects the m states back to the N stock dimension[cite: 95, 97, 98, 99, 100]. The output is then transposed back to its original shape.

Crucially, this market-influenced representation is combined with the original input H via a residual skip connection, not concatenation[cite: 96, 220]. The final output of the block, $\hat{H}$, is therefore an addition:

$$\hat{H} = H + \text{Transpose}(\text{MLP}(\text{Transpose}(H))) \in \mathbb{R}^{N \times h} \quad (3.7)$$

This market-aware representation $\hat{H}$ is then passed to a fully connected layer for the final next-day return prediction. This design, based on adding market-based residuals, captures both individual asset patterns and collective market dynamics.

3.5 Performance

In extensive experiments on three public benchmarks (NASDAQ, NYSE, and S&P 500), StockMixer outperforms prior state-of-the-art models across all major evaluation metrics (IC, RIC, PrecisionN, and Sharpe Ratio). As shown in (Figure 3.5), on NASDAQ StockMixer achieves an Information Coefficient (IC) of 0.043 and Rank Information Coefficient (RIC) of 0.501 surpassing the best hypergraph method (STHAN-SR: IC = 0.039, RIC = 0.451) and improving over classic graph attention (GAT: IC = 0.035, RIC = 0.377) and RNN baselines (LSTM: IC = 0.032, RIC = 0.354) . Preci-

	Model	NASDAQ				NYSE				S&P500			
		IC	RIC	prec@N	SR	IC	RIC	prec@N	SR	IC	RIC	prec@N	SR
RNN	LSTM	0.032	0.354	0.514	0.892	0.024	0.256	0.512	0.857	0.031	0.186	0.531	1.332
	ALSTM	0.035	0.371	0.522	0.941	0.023	0.276	0.519	0.764	0.029	0.181	0.532	1.298
GNN	RGCN	0.034	0.382	0.516	1.054	0.025	0.275	0.517	0.932	0.028	0.175	0.528	1.359
	GAT	0.035	0.377	0.530	1.233	0.025	0.297	0.521	1.070	0.034	0.191	0.541	1.484
	RSR-I	0.038	0.398	0.531	1.238	0.026	0.284	0.519	0.098	0.033	0.200	0.542	1.437
HGNN	STHAN-SR	0.039	0.451	0.543	1.416	0.029	0.344	0.542	1.228	0.037	0.227	0.549	1.533
	ESTIMATE	0.037	0.444	0.539	1.307	0.030	0.327	0.536	1.115	0.035	0.241	0.553	1.547
MLP	Linear	0.019	0.188	0.505	0.517	0.015	0.163	0.497	0.625	0.016	0.156	0.520	0.674
	StockMixer	0.043	0.501	0.545	1.465	0.029	0.351	0.539	1.454	0.041	0.262	0.551	1.586

Table 2: Comparison results on stock metrics (measured by t-test with p-value < 0.01). The methods for comparison are mainly divided into four types: RNN (Recurrent Neural Network), GNN (Graph Neural Network), HGNN (HyperGraph Neural Network) and MLP (Multi-Layer Perceptron). Bold & underlines show best & second best (SOTA) results, respectively.

Figure 3.3: Performance metrics reported by StockMixer

sionN and Sharpe Ratio likewise favor StockMixer (0.545 and 1.465) over all competitors.

On NYSE, one of the largest and most volatile markets, StockMixer attains an IC of 0.029 and RIC of 0.351, outperforming ESTIMATE (IC = 0.030, RIC = 0.327) and STHAN-SR (IC = 0.029, RIC = 0.344). Even in the smaller S&P 500 universe, StockMixer leads with an IC of 0.041 and Sharpe Ratio of 1.586, demonstrating robustness across differing market scales.

Relative to hybrid architectures, StockMixer’s unified MLP-based design yields an average gain of approximately 7.6% in IC, 10.8% in RIC, and 10.9% in Sharpe Ratio ($p < 0.01$), all while maintaining computational efficiency comparable to simple linear models. This consistent superiority across rank and return metrics establishes StockMixer as the new state of the art in stock price forecasting.

Chapter 4

Methodology

4.1 Problem definition

We follow the setup in StockMixer [5] by considering a stock market with N stocks. For each stock i , the historical data is represented as

$$X_i \in \mathbb{R}^{T \times F},$$

where T denotes the lookback window length and F is financial indicators and OHLC prices recorded at each time step. The overall input is thus given by

$$X \in \mathbb{R}^{N \times T \times F}.$$

The objective is to predict the closing price p_t^i for stock i on trading day t and subsequently compute the 1-day return ratio as

$$r_t^i = \frac{p_t^i - p_{t-1}^i}{p_{t-1}^i}.$$

Denoting the model parameters as θ , the entire forecasting process can be summarized as:

$$X \in \mathbb{R}^{N \times T \times F} \xrightarrow{\theta} p \in \mathbb{R}^{N \times 1} \rightarrow r \in \mathbb{R}^{N \times 1}.$$

4.2 Model overview

4.2.1 Input representation

Our model takes as input a feature matrix of shape $(N \times F)$, where each row corresponds to one asset and each column corresponds to an asset’s multi-scale temporal representation (Figure 4.1). In the case of the Stock-Mixer baseline, these features are obtained by concatenating multi-scale feature representations aggregated by previous mixing layers.

4.2.2 Indicators Used for Internal Gating Instead of Input Prices Augmentation

We compute market-level context from the closing-price time series. Specifically, the dataset must include close prices for all assets so that, over each 16-day window, we can calculate a suite of technical indicators and derived mathematical features such as mean return, realized volatility, dispersion, and principal-component dominance—that succinctly summarize prevailing market conditions.

Rather than concatenating these context features to the input price vectors, we inject them directly into the gating branches of our gMLP blocks (Figure 4.4). This design allows the model to dynamically modulate information flow based on market context, improving its ability to capture complex inter-stock relationships without inflating the primary feature dimension.

4.2.3 Objective output

The overall objective is to learn a mapping from the combined asset-feature matrix and market context, enabling the model to capture both asset-specific signals and inter-asset relationships. The integration of global con-

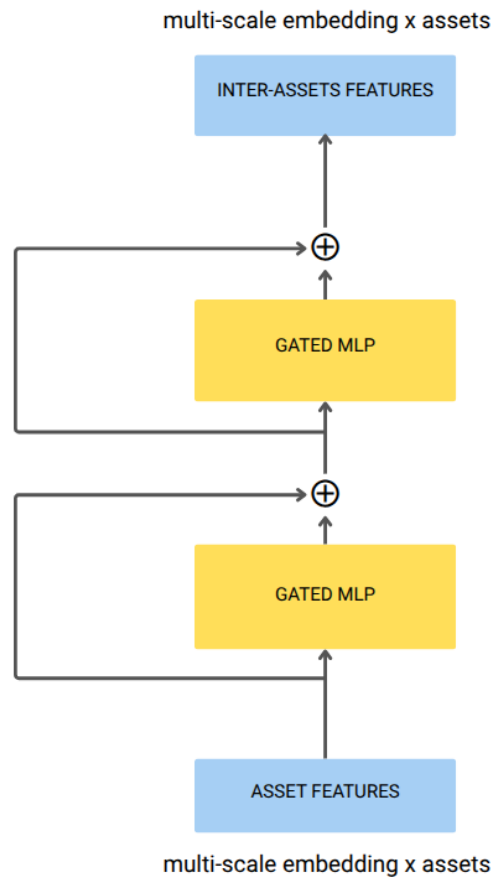


Figure 4.1: Stacked gated MLP for StockMixer

text further guides this interaction modeling in a data-driven and adaptive manner.

4.3 Extracting market context

We first convert closing prices to log returns over the 16-day lookback window, we define the log return of stock i at time t as:

$$r_{i,t} = \log(P_{i,t} / P_{i,t-1}).$$

Then compute five complementary features to form the market context tuple. These features are fed into the gating branch to consolidate and modulate the information flow, allowing the model to adapt its behavior to prevailing market conditions.

1. **Window Mean Return:** The average return across all stocks over the lookback window, indicating whether, on balance, prices in that window have trended up or down.

$$\text{MeanReturn} = \frac{1}{N W} \sum_{i=1}^N \sum_{t=1}^W r_{i,t}.$$

2. **Momentum Slope:** The slope of the equally-weighted index series through time within the window, quantifying both the direction (upward vs. downward) and the steepness of that trend.

$$\bar{r}_t = \frac{1}{N} \sum_{i=1}^N r_{i,t}, \quad \bar{r} = \frac{1}{W} \sum_{t=1}^W \bar{r}_t, \quad \bar{t} = \frac{W+1}{2}.$$

$$\text{Slope} = \frac{\sum_{t=1}^W (t - \bar{t}) (\bar{r}_t - \bar{r})}{\sum_{t=1}^W (t - \bar{t})^2}.$$

3. **Realized Volatility:** The standard deviation of the window's index

returns, measuring the typical magnitude of price swings—and thus the average “riskiness”—during the period.

$$\text{Volatility} = \sqrt{\frac{1}{W} \sum_{t=1}^W (\bar{r}_t - \bar{r})^2}.$$

4. **Dispersion:** The cross-sectional spread of individual stock returns (averaged across time), showing whether stocks move in unison (low dispersion) or pull apart (high dispersion).

$$\text{Dispersion} = \frac{1}{W} \sum_{t=1}^W \sqrt{\frac{1}{N} \sum_{i=1}^N (r_{i,t} - \bar{r}_t)^2}.$$

5. **PCA on the covariance matrix ($16 \times N$):** Compute the covariance of the N stock return series over the 16-day window, extract its first principal component’s eigenvalue, and divide by the total variance.

$$\text{PCARatio} = \frac{\lambda_{\max}}{\sum_{j=1}^N \lambda_j}, \quad \{\lambda_j\} = \text{eigenvalues of Cov}(r_{:,k:k+W-1}).$$

Finally, each feature is rescaled to $[0, 1]$ via min–max normalization before being passed to the gating branch.

4.4 Learning selectivity and modulating markets via a gating mechanism

Gated MLPs [10], present a streamlined architecture characterized by three key operations (Figure 4.2):

- **Channel Projection:** A standard MLP that projects each feature channel independently to enhance expressiveness.

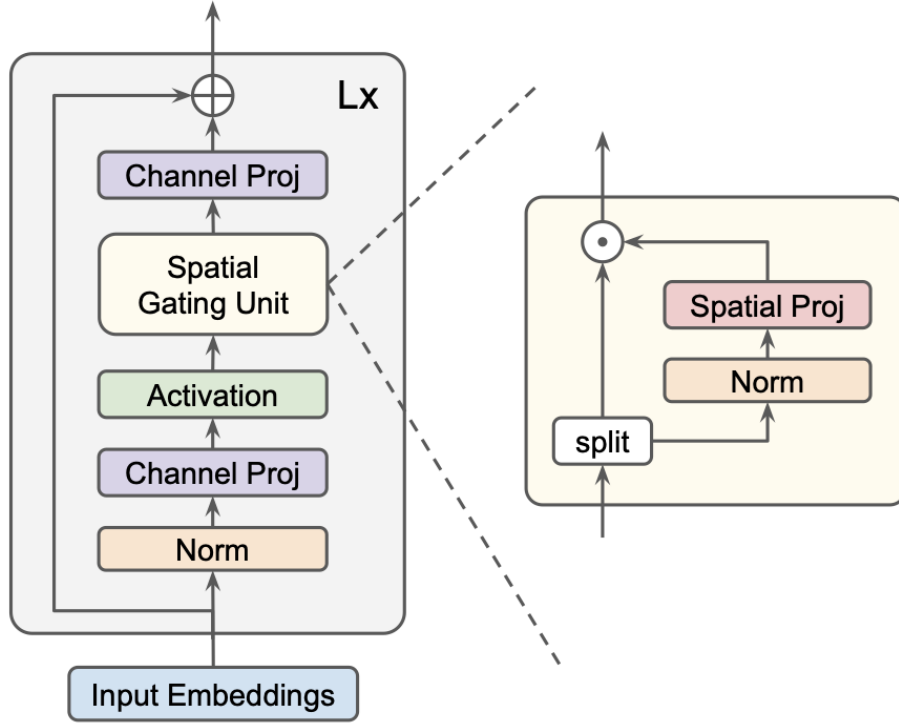


Figure 4.2: Gated multilayer perceptron architecture

- **Spatial Projection:** A depthwise linear transformation across the spatial or sequential dimension, capturing interactions between positions without full self-attention.
- **Gating Mechanism:** A lightweight branch that computes element-wise gates conditioned on the input, modulating the spatial projections to selectively emphasize or suppress features.

These operations are arranged in a parallel serial fashion within each layer. First, the channel projection and gating branches operate in parallel: the channel branch increases representational capacity, while the gating branch generates dynamic weights. Next, spatial projection applies position wise mixing under the control of those gates, realizing an efficient form of context aware feature aggregation. Around each gMLP block, residual connections and layer normalization ensure stable training and facilitate deeper stacking. By replacing attention with masked linear layers,

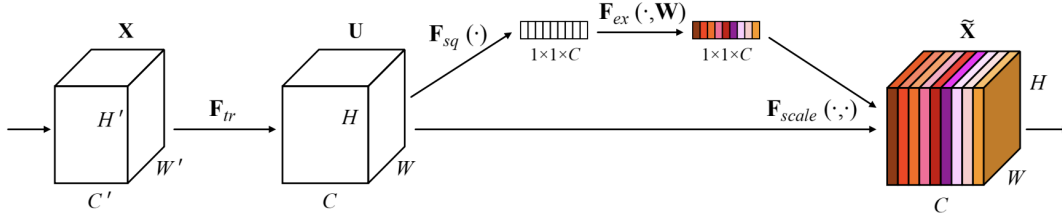


Figure 4.3: Squeeze-and-Excitation Block

gMLP achieves competitive performance with significantly reduced computational overhead, making it well-suited for both vision and sequential modeling tasks.

We further incorporate ideas from squeeze-and-excitation networks [11] (Figure 4.3), which introduce lightweight channel-wise gating by the use of the **sigmoid** function to ensure the resulting gate values lie strictly between 0 and 1, permitting each feature channel to be softly scaled attenuating less informative signals while amplifying those most pertinent to the downstream task.

As these gMLP blocks propagate through the network, the gating branch, conditioned on the precomputed market context, learns latent market states that dynamically modulate information flow by sigmoid activation, capturing complex behaviors such as cycles and regime shifts, while preserving the simplicity of a fully connected architecture.

4.5 Refined Gated MLP

To capture complex market-level interactions within a lightweight architecture, we refine the conventional gated MLP module with a specialized gating design (see Figure 4.4).

Furthermore, since the spatial projection in the gMLP block is less suitable when the spatial dimension is defined by concatenated multi scale time representations—and the preceding indicator and time mixing layers already capture rich spatial information, we omit spatial projection in this

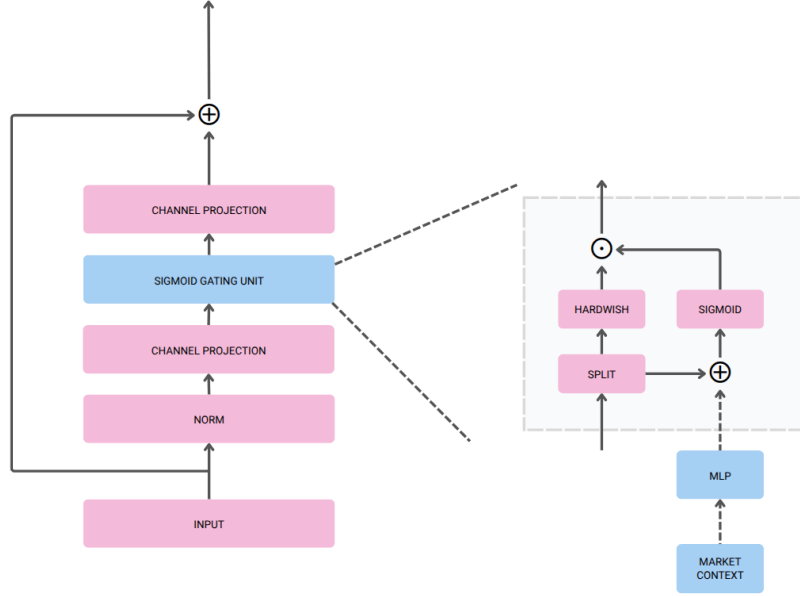


Figure 4.4: Novel gated MLP

module.

Our refinements are as follows:

1. **Channel projection and split:** A linear projection is first applied to compress the stock representations along the channel dimension. This creates a lower-dimensional 'hidden market' embedding, effectively introducing a bottleneck structure that enhances representation efficiency. We then split this compressed embedding into two equal parts, yielding two parallel branches.
2. **Gating Branch:** This branch incorporates external market context to generate dynamic gating weights. Specifically, we map the precomputed context vector through a small two-layer MLP (with GELU activation) to the same hidden dimension, add it to the branch's input, and apply a sigmoid activation. The result is a set of per-channel gating coefficients in $[0, 1]$ that selectively emphasize or suppress latent market features.

3. **Trunk Branch:** The second branch applies a non-linear activation (HardSwish) directly to its input channels. These activated features represent the “content” or candidate signals that may be passed on if deemed relevant by the gate.
4. **Fusion:** Finally, we multiply the trunk features by the sigmoid gating coefficients element-wise. This selects the dimensions carrying useful market information while filtering out noise. A final linear projection restores the original channel size, and a residual connection adds the result back to the input.

To better capture market-level signals, we formulate this design as follows:

Let

$$X \in \mathbb{R}^{d \times N}$$

be the input, where d is the concatenated multi-scale time dimension and N the number of stocks. We also have a precomputed market context vector

$$\text{ctx} \in \mathbb{R}^c.$$

1. LayerNorm and channel projection.

$$\tilde{X} = \text{LayerNorm}(X), \quad Y = \tilde{X} W_1, \quad Y \in \mathbb{R}^{d \times 2m},$$

where $W_1 \in \mathbb{R}^{N \times 2m}$ reduces channels to $2m$.

2. Split into trunk and gate inputs.

$$[Y_t, Y_g] = \text{chunk}(Y, 2), \quad Y_t, Y_g \in \mathbb{R}^{d \times m}.$$

3. Inject context into gating branch. First, map the context to the channel dimension by a MLP layer:

$$C = \text{GELU}(\text{ctx } W_c) W_d, \quad C \in \mathbb{R}^m,$$

where $W_c \in \mathbb{R}^{c \times \frac{m}{2}}$ and $W_d \in \mathbb{R}^{\frac{m}{2} \times m}$. Then broadcast-add:

$$Y'_g = Y_g + C.$$

4. Apply activations.

$$T = \text{HardSwish}(Y_t), \quad G = \text{Sigmoid}(Y'_g).$$

5. Merge branches and project back.

$$Z = T \odot G, \quad Z \in \mathbb{R}^{d \times m},$$

$$H = Z W_2, \quad H \in \mathbb{R}^{d \times N}, \quad W_2 \in \mathbb{R}^{m \times N}.$$

6. Residual connection.

$$X' = X + H.$$

With this design, the gating branch—conditioned on both the current stock embedding and external market context—learns which latent features to pass through, while the trunk branch provides the actual feature content. The element-wise multiplication then produces a context-aware filtering of market signals, all within a simple, attention-free MLP block.

This design promotes efficient feature reuse through residual learning. It also enhances interpretability by separating the influence of global market context from asset-specific signals. Each gated unit can be seen as a decision boundary that regulates which latent dimensions are allowed to influence downstream predictions, based on both local patterns and market-

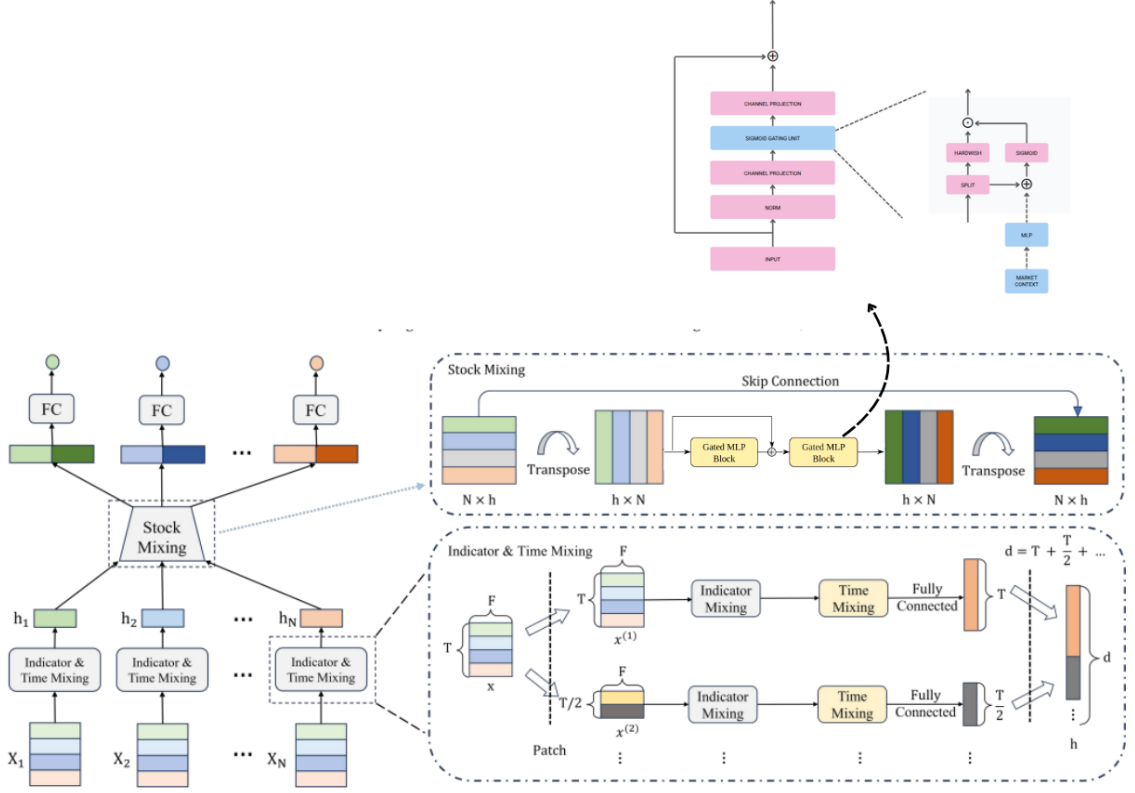


Figure 4.5: Gated MLP integration for StockMixer

wide behavior. From an information-theoretic perspective, this structure encourages the model to compress noisy input into a more salient latent space, guided by macro features.

As a result, the model remains robust across varying regimes, particularly when the input distribution exhibits non-stationarity or sharp transitions between market phases. This gating mechanism can thus be interpreted as a soft dynamic routing strategy that balances global and local information adaptively.

4.6 Enhance StockMixer approach

Our experiments focus exclusively on enhancing StockMixer’s stock-mixing module. Before this stage, StockMixer independently processes

each asset through indicator and time mixing layers, yields a mixed representation that captures both indicator-level and time-series patterns. These representations are concatenated into a matrix of shape $(N \times d)$, where N is the number of stocks and d is the concatenated scales features. This representation serves as the input to the stock mixing module, which further captures inter-stock relationships.

To enhance this module, we replace the original single-layer MLP with a stack of L gated MLP blocks (Figure 5.4), each equipped with residual connections, which augments standard MLP layers with conditional gating to selectively modulate information along the assets or temporal axes.

By stacking gated MLP layers, our approach enables progressively deeper refinement of cross-asset interactions without the use of attention mechanisms or graph structures. This iterative transformation sharpens the inter-stock representation at each layer, promoting the emergence of salient signals while filtering out noise. As a result, the model is able to capture rich asset relationships in a fast, scalable, and purely MLP-based manner.

Chapter 5

Experiments

5.1 Multi-timeframe

Initially, our efforts were directed toward enhancing the StockMixer’s multi-scale temporal modeling strategy. We explored several techniques, including integrating MSGNet’s dynamic time scale selection via FFT to rank salient temporal frequencies, employing discrete wavelet transforms (DWT) to decompose time series into multiple resolutions, and augmenting the multi-scale fusion layer with gating mechanisms to adaptively emphasize informative scales. Despite extensive hyperparameter tuning and empirical validation, these approaches consistently yielded suboptimal performance across key evaluation metrics (see Table 5.1). In the following sections, we briefly outline these attempted methodologies and report the corresponding experimental outcomes.

5.1.1 LagMixer: Embedding via Temporal Lag

As a first alternative, we replace StockMixer’s original patch embedding mechanism with a lag-based embedding strategy (Figure 5.1). This design aims to encode periodic temporal dependencies more explicitly by embedding fixed-lag time step pairs, rather than consecutive time steps.

Concretely, within a window of size 16, instead of embedding adjacent

Table 5.1: NASDAQ metrics for Dynamic Scale-Aware Embedding, Wavelet-Based Multi-Scale Embedding and Multiscale weighting

Hyperparameter	IC	RIC	pred@10	SR
Wavelet-Based Multi-Scale Embedding	0.017	0.207	0.522	1.69
Dynamic Scale-Aware Embedding	0.015	0.159	0.518	1.65
Multiscale weighting	0.020	0.257	0.527	0.270
Baseline	0.033	0.393	0.531	1.440

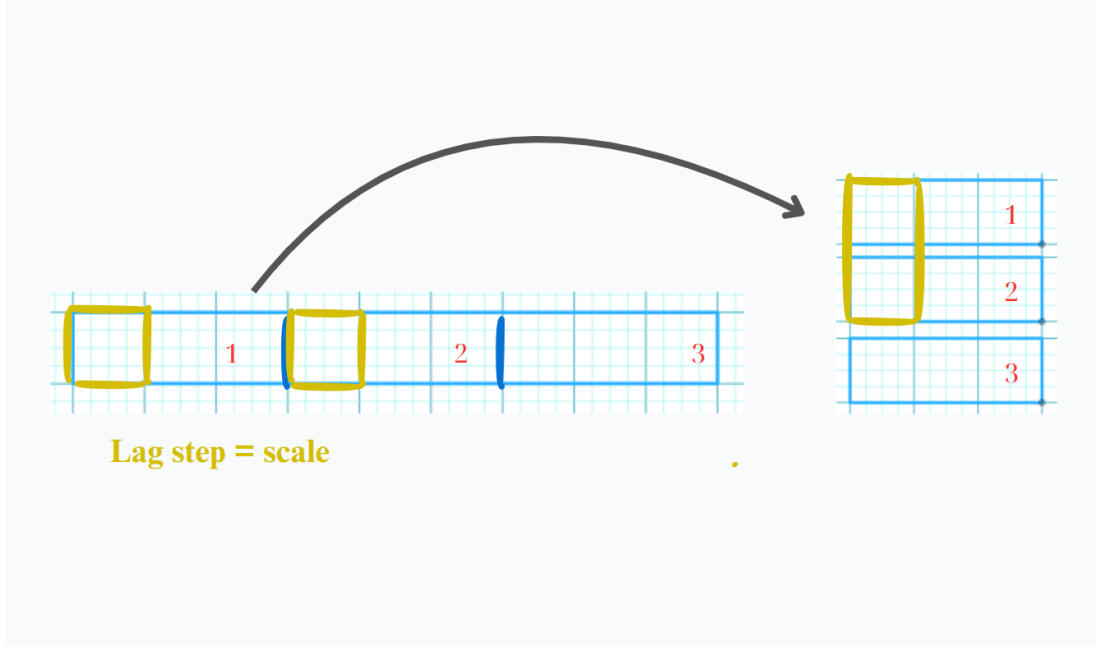


Figure 5.1: Experimental LagMixer

patches (e.g., the 1st and 2nd time steps, corresponding to a temporal scale of 8), we embed lagged pairs such as the 1st and 8th time steps. This approach is motivated by the intuition that periodic or cyclical market behaviors often manifest at fixed intervals, and a lag-based representation may better capture such recurring patterns.

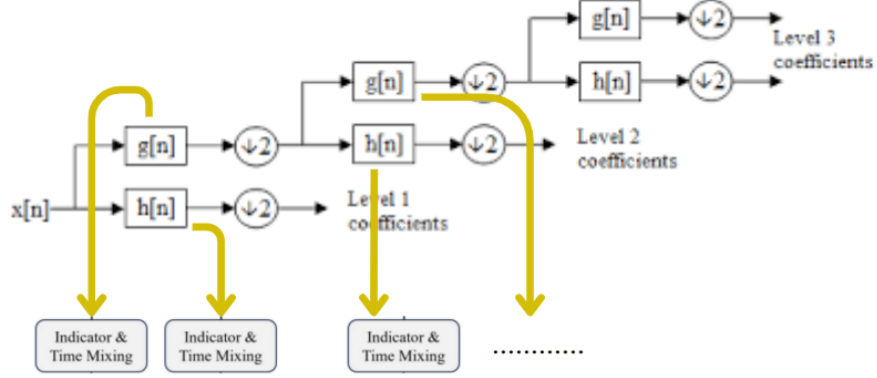


Figure 5.2: Experimental DWTMixer

5.1.2 DWTMixer: Wavelet-Based Multi-Scale Embedding

Inspired by the Multi-Scale Two-Way DNN (MTDNN) framework [1], we propose DWTMixer, a variant that substitutes StockMixer’s patch embedding with a hierarchical decomposition using the Discrete Wavelet Transform (DWT). While patch embedding implicitly performs a down-sampling operation over temporal sequences, DWT offers a more principled and interpretable alternative by progressively decomposing the input into approximation (cA) and detail (cD) coefficients at each scale (Figure 5.2).

These coefficients represent the low- and high-frequency components of the time series, respectively, capturing both long-term trends and short-term fluctuations. At each decomposition level, the corresponding cA and cD features are processed independently through dedicated mixing layers, allowing the model to learn scale-specific representation. The outputs from all scales are then aggregated following the standard fusion procedure in StockMixer.

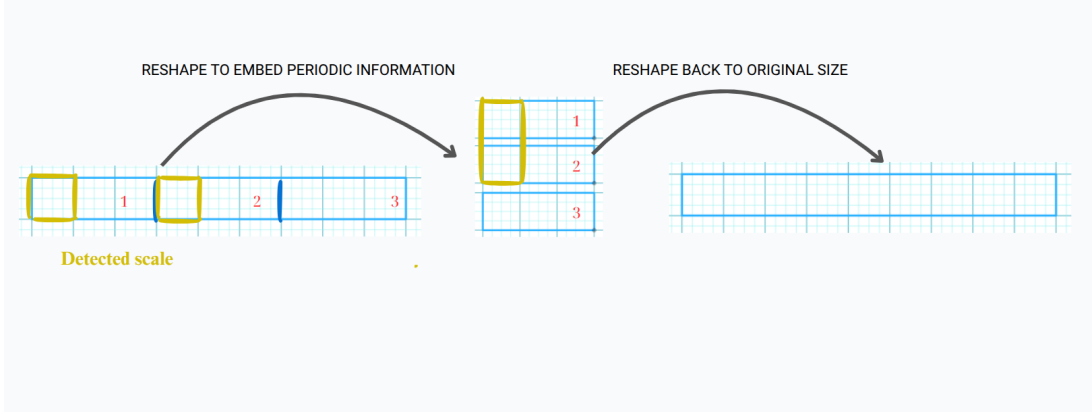


Figure 5.3: Experimental Dynamic Scale-Aware Embedding

5.1.3 Dynamic Scale-Aware Embedding

Building on MSGNet’s dynamic time-scale detection via Fast Fourier Transform (FFT) **cai2024msgnet**, we employ a Dynamic Scale-Aware Embedding module that identifies and exploits the most salient periodicities in the input sequence. First, we apply an FFT to each lookback window to compute the spectral power across frequencies and rank the resulting scales by their amplitude. Rather than downsampling, we reserve the original temporal resolution and use the top- k detected lags to reorder the window in the spirit of LagMixer (Figure 5.3), thereby embedding information at those critical intervals while preserving the input shape.

For each rank, we allocate a dedicated mixing layer to model its unique periodic behavior. Because the detected lags may not divide the window length evenly, we employ padding followed by truncation to maintain a fixed input size.

5.1.4 Multi-Scale Weighting

In parallel with our other enhancements, we investigated a lightweight attention-inspired mechanism to improve the scale fusion stage. Specifically, we introduced a gating module with sigmoid activation to learn the relative importance of each temporal scale. This module operates on the

Table 5.2: Detailed information on the number of assets, start and end dates, as well as the training, validation, and test durations for each dataset.

Attribute	NASDAQ	S&P 500	Crypto
Number of Assets	1,026	474	117
Start Date	2013-01-02	2016-01-04	2015-08-07
End Date	2017-12-08	2022-05-25	2018-06-06
Training Days	756	1,006	620
Validation Days	252	253	207
Test Days	273	352	208
Total Days	1,201	1,611	1,035

outputs of the individual mixer layers—after the representations have been semantically enriched—assigning soft weights to each scale before aggregation. The intuition was to allow the model to emphasize more informative resolutions while suppressing less relevant ones. However, despite its conceptual simplicity and ease of integration, this gating strategy produced no notable improvements in empirical performance across our benchmarks. (see Table 5.1).

5.2 Datasets

5.2.1 Benchmarks

We evaluate our model on three benchmarks: two equity markets (NASDAQ and S&P 500) and one cryptocurrency market. Table 5.2 provides detailed statistics for each dataset, including the number of assets, the time span covered, and the number of days allocated to each phase of model training and evaluation. The NASDAQ dataset includes 1,026 assets from 2013-01-02 to 2017-12-08, while the S&P 500 contains 474 assets from 2016-01-04 to 2022-05-25. The cryptocurrency dataset comprises 117 assets with aligned sequences covering 1,035 consecutive trading days.

These benchmarks offer a diverse set of financial environments, enabling

Table 5.3: Data quality metrics for the cryptocurrency.

Column	Valid	Mismatched	Missing	Unique
splug	100%	0%	0%	1,644
symbol	100%	0%	0%	1,605
name	100%	0%	0%	1,643
date	100%	0%	0%	1,865

robust analysis of forecasting performance across traditional equity and high-volatility digital asset markets.

5.2.2 Dataset Sources

We source traditional and digital asset datasets from two established repositories:

- First, the equity time series originate from the benchmark collection introduced by StockMixer [5], which comprises daily price and MA data for a diverse set of publicly traded firms and has seen widespread adoption in recent financial forecasting research.
- Second, for the cryptocurrency domain, we leverage the “crypto-markets.csv” dataset available on Kaggle,¹ which aggregates historical daily quotes for a broad spectrum of digital tokens.

5.2.3 Cryptocurrency Preprocessing

Data evaluation

The data were evaluated in terms of validity, mismatch, and missing-value rates across all primary feature columns. As reported in Table 5.3 and 5.4, the “splug,” “symbol,” “name,” “date”, “open”, “low”, “high” and “close”

¹<https://www.kaggle.com/code/residentmario/time-series-plotting-optional/input?select=crypto-markets.csv>

Table 5.4: Statistical summary and validation metrics for price-related columns.

Column	Valid	Mismatched	Missing	Mean	Std Deviation
open	100%	0%	0%	355	14k
high	100%	0%	0%	423	17.3k
low	100%	0%	0%	298	11.6k
close	100%	0%	0%	354	14k

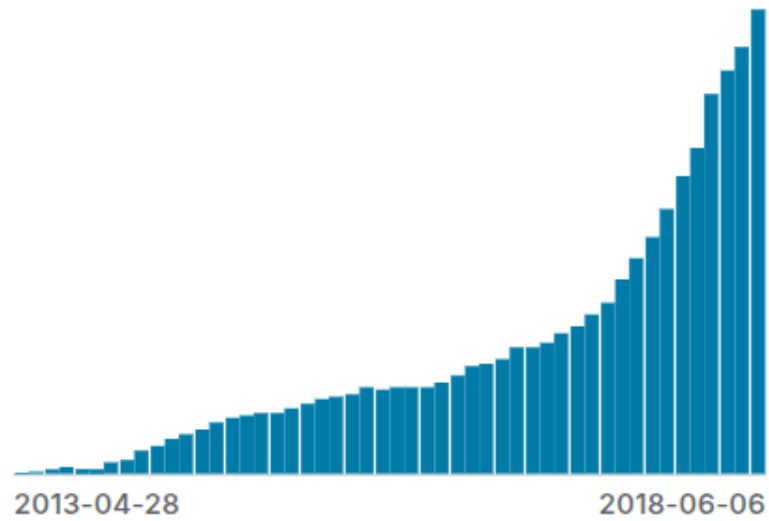


Figure 5.4: Cryptocurrency date distribution

fields each exhibit 100% validity with 0% mismatches and 0% missing entries. The date field ranges from a minimum of April 28, 2013 to a maximum of June 6, 2018, providing clear temporal bounds for the dataset.

Additionally, the dataset also included “close ratio,” and “spread” as auxiliary features; these were used solely for exploratory analysis and were not included in the model training pipeline. Continuous monitoring of these quality metrics confirms that the dataset maintains high integrity prior to model development.

Assets aggregating

The raw cryptocurrency dataset initially contains 1,644 assets, each exhibiting highly variable listing dates, lifespans, and trading activity profiles. Such heterogeneity poses two key challenges:

1. Constructing a consistent evaluation window that balances the desire for a large asset universe against the requirement for uninterrupted historical records.
2. Ensuring that the selected subset represents the most significant market participants.

To address these, we first identify the subset of tokens that share a sufficiently long common trading span. By examining listing and delisting dates, we isolate a contiguous 1,035-day interval—spanning August 7, 2015, through June 6, 2018 that maximizes coverage among the top traded cryptocurrencies. Next, we include all additional assets whose complete trading histories exactly cover this interval, irrespective of their individual market caps.

The resulting collection comprises 117 coins, each represented by daily OHLC and volume series over the identical 1,035 day horizon. This uniformity ensures that subsequent model comparisons are not undermined by heterogeneous time spans or irregular data gaps.

Indicators

From cryptocurrency, we include only the standard OHLC (open, high, low, close) price series together with corresponding trading volume indicator. To harmonize the dynamic range across assets and prevent scale disparities from biasing our models, every time series is independently normalized to the unit interval $[0, 1]$.

Training split

To emulate a realistic forecasting environment and eliminate any look-ahead bias, the normalized time series are then segmented in chronological order into training, validation, and test sets, adhering to a 60%/20%/20% temporal split.

5.3 Baselines

5.3.1 StockMixer as primary baseline

We conduct a comprehensive evaluation of our gated MLP integration by first benchmarking it against the StockMixer architecture, which serves as our primary baseline. All models are trained and tested under identical data splits and configurations including a fixed learning rate and a 16-day lookback window—to ensure that observed performance differentials arise solely from architectural innovations.

For the cryptocurrency dataset, we re-ran the StockMixer baseline using the reported experimental scale factors $\{1, 2, 4\}$ and market hidden dimension values up to 40. We found that a scale factor of 2 and a hidden dimension of 10 yielded the best test performance.

5.3.2 Additional state of the art models

To further broaden the comparative scope of our empirical study, we include three well-established neural sequence modeling architectures as additional baselines. Specifically, we evaluate

1. A vanilla Long Short-Term Memory (LSTM) network, which has long been a cornerstone of sequence learning due to its ability to mitigate vanishing gradients
2. A Gated Recurrent Unit (GRU) network, which offers a more parameter efficient recurrent alternative while retaining the gating mechanisms critical for capturing long-range dependencies.
3. A standard Transformer model, whose self-attention mechanism has revolutionized sequence-to-sequence tasks by enabling direct modeling of pairwise token interactions.

By aligning these recurrent and attention-based frameworks against our proposed method, we gain deeper insight into the relative strengths and limitations of each architectural paradigm.

5.4 Metrics

To facilitate direct comparison with prior work, we reuse the four evaluation metrics, as follows:

1. Information Coefficient (IC), measuring the Pearson correlation between predicted and actual returns, captures pointwise predictive accuracy.
2. Rank Information Coefficient (RIC), computed via Spearman rank correlation, quantifies the preservation of relative ranking among assets.

Table 5.5: Empirically tuned hyperparameter settings across NASDAQ, S&P 500, and Crypto datasets.

Hyperparameter	NASDAQ	S&P 500	Cryptocurrency
market hidden size	32	15	20
number of gMLP blocks	1	3	5

3. Precision@N, the fraction of top-N predicted assets that realize positive returns, evaluates the model’s practical asset-selection efficacy.
4. Sharpe Ratio (SR), the risk-adjusted return metric, assesses profitability normalized by return volatility.

These well-established metrics comprehensively assess both predictive accuracy and ranking quality, enabling fair comparison with existing state-of-the-art methods while eliminating the need to restate their formulations.

5.5 Implementation Details

Our models are implemented in PyTorch. To ensure a fair comparison with the baseline model, we use a 16-day lookback window and maintain the temporal scale factors as provided in the prior work. We determine the hyperparameter settings as Table 5.5 based on empirical tuning, while keeping the learning rate unchanged.

To further guarantee a controlled evaluation, we re-evaluated the baseline model and our integration on the same hardware configuration. Although the original code was executed on a different GPU setup (e.g., Nvidia GeForce RTX 2080 Ti), our experiments were conducted on a consistent platform (Kaggle GPU T4 x2). This controlled environment ensures that any differences in the reported metric values stem from model performance rather than variations in computational resources.

Moreover, since our models do not incorporate random parameters, the GPU T4 x2 produces consistent results on every run, eliminating the need for multiple trials and averaging.

Table 5.6: Comparison of models across NASDAQ, S&P 500, and Crypto datasets. Metrics: Information Coefficient (IC), Rank IC (RIC), precision@10 (prec@10), and Sharpe Ratio (SR).

Model	NASDAQ				S&P 500				Crypto			
	IC	RIC	prec@10	SR	IC	RIC	prec@10	SR	IC	RIC	prec@10	SR
LSTM	0.027	0.310	0.502	-1.160	0.012	0.166	0.534	2.240	-0.012	-0.130	0.498	4.000
GRU	0.026	0.263	0.515	-0.097	0.005	0.049	0.522	1.110	-0.003	-0.052	0.537	1.910
Transformer	0.017	0.258	0.503	-0.548	0.022	0.233	0.527	1.600	0.005	0.068	0.514	1.730
Baseline	0.033	0.393	0.531	1.440	0.021	0.150	0.527	1.920	0.023	0.262	0.525	4.780
Ours	0.042	0.480	0.541	1.590	0.039	0.279	0.540	1.650	0.028	0.291	0.516	5.640

We further observed that the integration of gated MLP blocks introduces only a modest increase in architectural complexity, as each block is composed of lightweight operations combined with residual connections. The use of context-aware modules is carefully designed to preserve the memory efficiency and scalability of the original StockMixer architecture. This ensures that the overall parameter size and training resource requirements remain practical for mid-range GPU setups.

During training, gradient propagation through residual paths remained stable across datasets, and the model demonstrated consistent convergence behavior under fixed learning schedules. To ensure reproducibility, we repeated each experiment three times using fixed random seeds and observed minimal variance in final Information Coefficient and Sharpe Ratio values. These observations highlight the robustness and training reliability of the proposed design, even without the need for extensive tuning or computational overhead.

5.6 Overall comparison

5.6.1 Re-evaluation of StockMixer

All experiments for the StockMixer baseline were conducted using the publicly available implementation on Kaggle’s GPU T4 x2 configuration with the hyperparameters as specified in the prior work.

Although the authors indicated exploration of scale factors up to 4, we determined that the released code employs a scale factor of 2 for NASDAQ, which also yields superior performance relative to scale factor 4 under our experimental conditions. For the S&P500 dataset, a scale factor of 4 achieves the best results among the tested values. We observed performance of the model that fell short of the high performance metrics claimed [5].

Nonetheless, our integration attains comparable outcomes. For instance, on the S&P500 benchmark, our model achieves an Information Coefficient (IC) of 0.039, a Rank Information Coefficient (RIC) of 0.279, and a Sharpe Ratio (SR) of 1.65, versus the study claimed values of 0.041, 0.262, and 1.586, respectively.

5.6.2 Results

The empirical results reported in Table 5.6 provide strong evidence that our gated-MLP integration yields significant performance gains over the original StockMixer architecture as well as over three standard sequence modeling approaches—namely, LSTM, GRU, and Transformer—across all evaluated domains. Specifically, the enriched stock mixing mechanism introduced by our gated MLP blocks consistently enhances key performance indicators, including predictive correlation (IC), ranking fidelity (RIC), and risk adjusted returns (Sharpe Ratio). These improvements underscore the efficiency of our approach in learning richer inter-asset relationships while preserving computational efficiency.

5.6.3 Discussion and Remarks

Key performance indicators improvements

Although the differences in Precision@10 are relatively modest, the consistent gains observed in Information Coefficient (IC) and Rank Infor-

mation Coefficient (RIC) indicate that our model extension significantly improves both the predictive accuracy and the robustness of the forecasting framework. In particular, the improvements in IC and RIC reflect a more accurate and effective ranking of stocks based on their latent return potential, while the enhanced SR on cryptocurrency and NASDAQ demonstrates that our model achieves superior risk-adjusted returns.

Cryptocurrency noisy nature exhibited

Notably, the cryptocurrency data consistently exhibited higher errors, reflecting the inherently noisy and volatile nature of digital assets. As a result, the Information Coefficient achieved on the crypto benchmark remains lower than those observed for NASDAQ and S&P 500. Nevertheless, our model still attains the best IC among all baselines in this setting, demonstrating its robustness and adaptability to highly unstructured, high-volatility markets.

Collectively, these findings underscore that our approach significantly augments the expressive power of the purely MLP-based forecasting model. It enables StockMixer to learn better inter-asset dependencies, thereby producing more accurate predictions and robust rankings than both its original form and established recurrent or attention-based alternatives.

5.7 Ablation study

5.7.1 Model Component

To assess the specific contribution of market context in our gated-MLP architecture, we conduct an ablation study that disables the context enhancement. Specifically, we remove the window-level context vector from the gating branch in each gMLP block, effectively preventing the model from accessing global market statistics such as mean return, realized volatility, and dispersion (see Figure 5.5).

Table 5.7: Comparison of Baseline, No Context, and With Context settings across NASDAQ, S&P 500, and Crypto datasets. Metrics: Information Coefficient (IC), Rank IC (RIC), precision@10 (prec@10), and Sharpe Ratio (SR).

Market	Baseline				No Context				With Context			
	IC	RIC	prec@10	SR	IC	RIC	prec@10	SR	IC	RIC	prec@10	SR
NASDAQ	0.033	0.393	0.531	1.44	0.037	0.501	0.549	1.55	0.042	0.480	0.541	1.59
S&P 500	0.021	0.150	0.527	1.92	0.004	0.030	0.507	0.34	0.039	0.279	0.540	1.65
Crypto	0.023	0.262	0.525	4.78	0.015	0.200	0.534	5.55	0.028	0.291	0.516	5.64

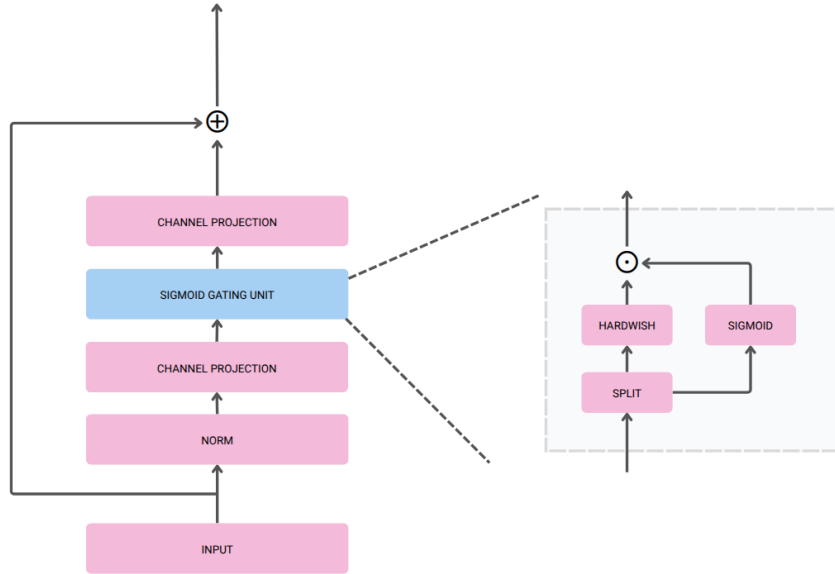


Figure 5.5: Ablated gated MLP

In this ablated variant, the gates operate exclusively on asset-specific feature embeddings, relying only on local information without any conditioning on broader market behavior. This modification allows us to directly assess how much contextual input contributes to learning richer representations and improving predictive performance.

5.7.2 Effect of ablated variants

The ablation results presented in Table 5.7 indicate that incorporating window-level market context—comprising mean return, momentum slope, realized volatility, dispersion, and PCA-derived variance—consistently improves both predictive correlation metrics (IC and RIC) and risk-adjusted return (Sharpe Ratio) across all evaluated domains: NASDAQ, S&P 500, and cryptocurrency. The removal of these contextual features leads to performance degradation, with no strong instances of improvement, thereby underscoring their universal utility in enhancing representation quality by capturing broader market dynamics.

NASDAQ almost unchanged

Notably, on NASDAQ the benefits of context integration are modest (e.g., IC +0.005, SR +0.04), implying only incremental gains in a market driven by broad economic or by the high noise levels characteristic of cryptocurrencies. By contrast, the S&P 500 and cryptocurrency benchmarks see much larger improvements with added context. This contrast indicates that window-level context features excel in capturing regime shifts and market-wide signals in diversified indices like the S&P 500.

Cryptocurrency degradation

Also, the use of market context features leads to even better results in the cryptocurrency data set. This may be because cryptocurrency markets

are more volatile, noisy, and less structured. In such unstable environments, simple global indicators, such as average return, volatility, or price differences between assets, can help the model make better decisions by providing more stable signals.

While stock prices are often influenced by industry or economic trends, cryptocurrencies tend to move based on online sentiment or sudden events, so having an overall market context is especially helpful for improving predictions and reducing noise.

In general, incorporating cross-sectional statistics offers a lightweight yet effective enhancement to temporal and inter-asset representations in MLP-based forecasting models.

Chapter 6

Conclusion And Future Works

6.1 Conclusion

In this paper, we introduce a novel gated MLP architecture for adaptive multi-scale stock price forecasting that combines multi-scale input segmentation with market context integrated gating to capture complex market dynamics. By comparing our model to the baseline, we show that its internal gating mechanism learns rich latent market representations enabling more robust inter-stock information exchange without relying on attention or graph layers.

Extensive experiments on a cryptocurrency dataset further demonstrate that our approach achieves superior performance particularly in Information Coefficient and Rank Information Coefficient while remaining lightweight and efficient. These results underscore the potential of attention- and graph-free MLP architectures for financial time series forecasting.

The results also suggest that context-aware gating mechanisms may play a broader role in financial modeling beyond simple return forecasting. For instance, this architecture could be extended to support classification-based tasks such as directional prediction or volatility clustering, by adjusting the output head. Moreover, the modular nature of the gMLP blocks enables easy integration into existing Transformer-free pipelines, which is

desirable in latency-sensitive applications such as high-frequency trading. Finally, the explicit separation of market context and asset-level signal paths opens promising directions for explainable AI (XAI), where each decision could be traced back to interpretable context triggers.

6.2 Future Works

We introduced a context-aware gating mechanism within an MLP framework to dynamically adjust asset features based on recent market behavior, leading to improved predictive accuracy and stability. Although our approach outperforms baseline models, it also suggests new opportunities for refinement and extension:

- **Expanded Context Features:** Incorporate additional market signals (e.g., sector-level indices, macroeconomic indicators) or alternative statistical summaries (e.g., skewness, kurtosis) to enrich the gating input.
- **Multi-Window Gating:** Extend the single 16-day lookback to a multi-window scheme (e.g., short, medium, long horizons) and learn hierarchical gating weights to capture dynamics at different scales.
- **Cross-Market Evaluation:** Validate the model on other asset classes (e.g., commodities, Forex) or non-U.S. equity markets to assess generalization across diverse trading environments.
- **Adaptive Hyperparameter Search:** Use straightforward tuning methods such as Bayesian optimization or genetic algorithms to automatically select gating network parameters with minimal manual effort.
- **Lightweight Attention Hybrids:** Investigate combining our context-aware gating with sparse or linearized attention layers to selectively

incorporate fine-grained inter-asset interactions while keeping complexity low.

Submitted Papers

1. PRICAI 2025(Rank B) Under Review

References

- [1] Liu, G., Mao, Y., Sun, Q., *et al.*, “Multi-scale two-way deep neural network for stock trend prediction,” in *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, 2021, pp. 4555–4561.
- [2] Cai, W., Liang, Y., Liu, X., Feng, J., and Wu, Y., “Msgnet: Learning multi-scale inter-series correlations for multivariate time series forecasting,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 10, pp. 11 141–11 149, Mar. 2024. DOI: 10.1609/aaai.v38i10.28991. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/28991>.
- [3] Sawhney, R., Agarwal, S., Wadhwa, A., Derr, T., and Shah, R. R., “Stock selection via spatiotemporal hypergraph attention network: A learning to rank approach,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 1, pp. 497–504, May 2021. DOI: 10.1609/aaai.v35i1.16127. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/16127>.
- [4] Oreshkin, B. N., Carpo, D., Chapados, N., and Bengio, Y., “N-beats: Neural basis expansion analysis for interpretable time series forecasting,” *arXiv preprint arXiv:1905.10437*, 2019.
- [5] Fan, J. and Shen, Y., “Stockmixer: A simple yet strong mlp-based architecture for stock price forecasting,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 8, pp. 8389–8397, Mar. 2024. DOI: 10.1609/aaai.v38i8.28681. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/28681>.
- [6] Carta, S., Ferreira, A., Podda, A. S., Reforgiato Recupero, D., and Sanna, A., “Multi-dqn: An ensemble of deep q-learning agents for stock market forecasting,” *Expert Systems with Applications*, vol. 164, p. 113 820, 2021, ISSN: 0957-4174. DOI: <https://doi.org/10.1016/j.eswa.2021.113820>.

- 1016/j.eswa.2020.113820. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417420306321>.
- [7] Liu, X. and Wang, W., “Deep time series forecasting models: A comprehensive survey,” *Mathematics*, vol. 12, no. 10, 2024, ISSN: 2227-7390. DOI: 10.3390/math12101504. [Online]. Available: <https://www.mdpi.com/2227-7390/12/10/1504>.
 - [8] Zeng, A., Chen, M., Zhang, L., and Xu, Q., “Are transformers effective for time series forecasting?” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 9, pp. 11 121–11 128, Jun. 2023. DOI: 10.1609/aaai.v37i9.26317. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/26317>.
 - [9] Chen, S.-A., Li, C.-L., Yoder, N., Arik, S. O., and Pfister, T., *Tsmixer: An all-mlp architecture for time series forecasting*, 2023. arXiv: 2303.06053 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2303.06053>.
 - [10] Liu, H., Dai, Z., So, D., and Le, Q. V., “Pay attention to mlps,” *Advances in neural information processing systems*, vol. 34, pp. 9204–9215, 2021.
 - [11] Hu, J., Shen, L., and Sun, G., “Squeeze-and-excitation networks,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 7132–7141.
 - [12] Luo, Y., Hu, J., Wei, X., Fang, D., and Shao, H., “Stock trends prediction based on hypergraph modeling clustering algorithm,” in *2014 IEEE International Conference on Progress in Informatics and Computing*, 2014, pp. 27–31. DOI: 10.1109/PIC.2014.6972289.
 - [13] Li, W., Bao, R., Harimoto, K., Chen, D., Xu, J., and Su, Q., “Modeling the stock relation with graph network for overnight stock movement prediction,” in *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*, Bessiere, C.,

- Ed., Special Track on AI in FinTech, International Joint Conferences on Artificial Intelligence Organization, Jul. 2020, pp. 4541–4547. DOI: 10.24963/ijcai.2020/626. [Online]. Available: <https://doi.org/10.24963/ijcai.2020/626>.
- [14] Sawhney, R., Agarwal, S., Wadhwa, A., and Shah, R. R., “Spatiotemporal hypergraph convolution network for stock movement forecasting,” in *2020 IEEE International Conference on Data Mining (ICDM)*, 2020, pp. 482–491. DOI: 10.1109/ICDM50108.2020.00057.
 - [15] Wu, H., Xu, J., Wang, J., and Long, M., *Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting*, 2022. arXiv: 2106.13008 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2106.13008>.
 - [16] Xia, H., Ao, H., Li, L., *et al.*, “Ci-sthpan: Pre-trained attention network for stock selection with channel-independent spatio-temporal hypergraph,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 8, pp. 9187–9195, Mar. 2024. DOI: 10.1609/aaai.v38i8.28770. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/28770>.
 - [17] Lim, B. and Zohren, S., “Time-series forecasting with deep learning: A survey,” *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 379, no. 2194, p. 20200209, Feb. 2021, ISSN: 1471-2962. DOI: 10.1098/rsta.2020.0209. [Online]. Available: <http://dx.doi.org/10.1098/rsta.2020.0209>.
 - [18] Zhou, T., Ma, Z., Wen, Q., Wang, X., Sun, L., and Jin, R., *Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting*, 2022. arXiv: 2201.12740 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2201.12740>.

- [19] Zheng, C., Fan, X., Wang, C., and Qi, J., *Gman: A graph multi-attention network for traffic prediction*, 2019. arXiv: 1911.08415 [eess.SP]. [Online]. Available: <https://arxiv.org/abs/1911.08415>.
- [20] Zong, C., Wang, C., Qin, M., Feng, L., Wang, X., and An, B., *Macrohft: Memory augmented context-aware reinforcement learning on high frequency trading*, 2024. arXiv: 2406.14537 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2406.14537>.
- [21] Li, T., Liu, Z., Shen, Y., Wang, X., Chen, H., and Huang, S., “Master: Market-guided stock transformer for stock price forecasting,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 1, pp. 162–170, Mar. 2024. DOI: 10.1609/aaai.v38i1.27767. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/27767>.
- [22] Tolstikhin, I., Houlsby, N., Kolesnikov, A., *et al.*, *Mlp-mixer: An all-mlp architecture for vision*, 2021. arXiv: 2105.01601 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2105.01601>.